

数据结构与算法 实验报告

第 二 次



姓名 陈宇昂

班级 数学强基 2202 班

学号 2224110096

电话 **

Email yuang.chen2024@outlook.com

日期 2024-11-16

目录

任务 1: 为指定的 List ADT 实现各种数据结构	3
一、 题目	3
二、 算法设计	3
三、 主干代码说明	4
四、 运行结果展示	19
五、 总结与收获	22
任务 2: 为指定的 DQueue ADT 实现各种数据结构	23
一、 题目	23
二、 算法设计	23
三、 主干代码说明	23
四、 运行结果展示	28
五、 总结与收获	28
任务 3: 栈	29
子任务一:	29
一、 题目: 非递归快速排序	29
二、 数据结构设计	29
三、 算法设计	29
四、 主干代码说明	29
五、 运行结果展示	30
六、 总结与收获	30
子任务二:	31
一、 题目: 混合运算表达式	31
二、 数据结构设计	31
三、 主干代码说明	31
四、 运行结果展示	32
五、 总结与收获	33
子任务三:	34
一、 题目: 股票最大走势区间	34
二、 算法设计	34
三、 复杂度分析	35
四、 主干代码说明	35
五、 运行结果	36
六、 总结与收获	36
任务 4: 基数排序	37
一、 题目	37
二、 数据结构设计	37
三、 算法设计	37
四、 主干代码说明	37
五、 运行结果展示	41
六、 总结与收获	42

任务 1：为指定的 List ADT 实现 各种数据结构

一、题目

在本次实验中，主要完成的任务是：

1、为指定的 List ADT（该 ADT 的具体要求请参见文件 List.java）实现三种数据结构：①使用顺序数组做为存储表示；②使用单向链表做为存储表示；③使用双向链表做为存储表示。不论使用哪种存储表示实现的数据结构，实验中需要处理的数据类型都是 Character 类型。

2、用给定的输入数据文件验证所实现的数据结构是否正确。

3、给出每种数据结构实现的每个方法在最坏情形下的时间复杂度表示。（使用表格列举结论）为了方便进行测试验证，对 List 的各种操作指定了相应的命令符号，每个符号含义如下：

+	insert
-	remove
=	replace
#	gotoBeginning
*	gotoEnd
>	gotoNext
<	gotoPrev
~	clear

假如对初始化空间大小为 16 的空表按顺序依次执行每一行的命令列表，并且假设该线性表中插入的元素为 Character 类型，那么表格中第二列即为执行相对应的第一列中的命令列表后调用 showStructure 方法的运行结果（在阅读下面表格内容之前先仔细阅读 List.java 中对每个方法的详细说明，下一行的命令内容是接着上一行的命令内容执行）：

命令行内容	执行结果（调用 List 的 showStructure 方法） ¹
+a +b +c +d	a b c d {capacity = 16, length = 4, cursor = 3}
# >>	a b c d {capacity = 16, length = 4, cursor = 2}
* <<	a b c d {capacity = 16, length = 4, cursor = 1}
-	a c d {capacity = 16, length = 3, cursor = 1}
+e +f +f	a c e f f d {capacity = 16, length = 6, cursor = 4}
* -	a c e f f {capacity = 16, length = 5, cursor = 0}
-	c e f f {capacity = 16, length = 4, cursor = 0}
=g	g e f f {capacity = 16, length = 4, cursor = 0}
~	Empty list {capacity = 16, length = 0, cursor = -1}

实验完成之后，必须通过实验中提供的测试用例。借助测试用例的运行结果，用来检查所撰写

¹ 如果 List 实现的存储方式为链式结构，那么 capacity 的值即为真实的链表中的结点个数。实验文件中仅提供了基于数组实现的测试用例的运行结果。

的代码功能是否正确。测试用例中的每一行的内容都类似于上表中的每一行“命令行内容”列中所指示的内容。要求每执行一行，就调用 List 接口中的 showStructure 行为，用以验证该命令行的执行是否正确。每行“命令行内容”都不是独立的，是针对同一个 List 类型的对象实例运行的结果。实验包里包括了个文件，一个是“list_testcase.txt”，其内包含了测试用例；另一个是“list_result.txt”，其内包含了对应测试用例的运行结果。

二、算法设计

根据题给内容形成接口，完成单向、双向链表所依赖的单向、双向结点类，而后再形成实现题给接口的各线性表。为了进行题目要求的命令行操作，考虑形成一个测试类，用它来实现字符读取、根据字符进行线性表操作的功能，并且要实现对文件进行读取、对结果进行正确性检测的功能，由此来方便测试。

虽然所要处理的数据全都是 `Character` 类型，但是我们仍旧运用 `java` 中的泛型进行数据结构的搭建。

三、主干代码说明

(一) AList 类

```
1 package task1and2;
2
3 public class AList<T> implements List<T> { 1个用法
4     public static final int defaultsSize = 188; 1个用法
5     public int msize; 6个用法
6
7     public int numInList; 25个用法
8     public int curr; 30个用法
9     private Object[] listArray; 13个用法
10    > public AList() { this(defaultsSize); }
13    public AList(int msize) { 1个用法
14        this.msize = msize;
15        this.curr = 0;
16        this.numInList = 0;
17        listArray = new Object[msize];
18    }
```

1、insert

```
19
20 @Override 9个用法
21 public void insert(T newElement) throws ListException {
22     if(isFull()){
23         //System.out.println("List is Full");
24         return;
25     }
26     if(this.isEmpty()){
27         listArray[0] = newElement;
28         numInList++;
29         return;
30     }
31     curr++;
32     for(int i = numInList; i > curr; i--)
33         listArray[i] = listArray[i-1];
34     listArray[curr] = newElement;
35     numInList++;
36 }
```

2、remove

```
36
37 @Override 6个用法
38 public void remove() {
39     if(this.isEmpty()){
40         System.out.println("List is empty");
41         return;
42     }
43     if((curr < 0) && (curr >= numInList)) {
44         System.out.println("Bad value for curr while remove");
45         return;
46     }
47     for(int i = curr; i < numInList-1; i++)
48         listArray[i] = listArray[i+1];
49     if(curr == numInList - 1)
50         curr = 0;
51     numInList--;
52 }
```

3、replace

```
53
54 @Override
55 public void replace(T newElement) {
56     if((curr < 0) && (curr >= numInList)) {
57         System.out.println("Bad value for curr while replace");
58         return;
59     }
60     listArray[curr] = newElement;
61 }
```

4、clear

```

62      @Override 6个用法
63 ④ >      public void clear() {
64              listArray = new Object[msize];
65              curr = 0;
66              numInList = 0;
67      }

```

5、isEmpty 与 isFull

```

69      @Override
70 ④ >      public boolean isEmpty() { return numInList == 0; }
73
74      @Override 2个用法
75 ④ >      public boolean isFull() { return numInList == msize; }

```

6、四个 goto

```

79      @Override 6个用法
80 ④ >      public boolean gotoBeginning() {
81              if(this.isEmpty()){
82                  return false;
83              }
84              curr = 0;
85              return true;
86      }
87
88      @Override 6个用法
89 ④ >      public boolean gotoEnd() {
90              if(this.isEmpty())return false;
91              curr = numInList - 1;
92              return true;
93      }
94
95      @Override 6个用法
96 ④ >      public boolean gotoNext() {
97              if((curr >= 0)&&(curr < numInList - 1)){
98                  curr++;
99                  return true;
100             }
101             else return false;
102         }
103
104      @Override 6个用法
105 ④ >      public boolean gotoPrev() {
106              if((curr > 0)&&(curr < numInList)){
107                  curr--;
108                  return true;
109              }
110              else return false;
111         }

```

7、getCursor

```

113      @Override 0个用法
114 ④ >      public T getCursor() { return (T) listArray[curr]; }

```

8、showStructure

```

118      @Override 7个用法
119 ④ >      public void showStructure() {
120              if(this.isEmpty()){
121                  System.out.println("Empty list "+ "{capacity = " + msize + ", length = " + numInList + ", cursor = " + (curr-1) + "}");
122                  return;
123              }
124              for(int i = 0; i < numInList; i++){
125                  System.out.print(listArray[i]);
126                  System.out.print(' ');
127              }
128              System.out.println("{capacity = " + msize + ", length = " + numInList + ", cursor = " + curr + "}");// 00\000
129         }

```

9、moveToNth

```

131     @Override 0个用法
132     public void moveToNth(int n) throws ListException {
133         if (isEmpty() || n < 0 || n >= numInList) {
134             throw new ListException("Invalid index");
135         }
136         curr = n;
137     }

```

10、Find

```

139     @Override 0个用法
140     public boolean find(T searchElement) throws ListException {
141         if (isEmpty()) {
142             System.out.println("Invalid index");
143         }
144         int initial = curr;
145         for (int i = initial; i < numInList; i++) {
146             if (listArray[i].equals(searchElement)) {
147                 curr = i;
148                 return true;
149             }
150         }
151         curr = numInList - 1;
152         return false;
153     }

```

11、toString

```

155     public String toString() {
156         StringBuilder tmpString = new StringBuilder();
157         if (this.isEmpty()) {
158             tmpString = new StringBuilder("Empty list ");
159             tmpString.append("capacity = " + 512 + ", length = " + this.numInList + ", cursor = " + (this.curr - 1) + "}");
160             return tmpString.toString();
161         }
162         for (int i = 0; i < numInList; i++) {
163             tmpString.append(listArray[i]);
164             tmpString.append(' ');
165         }
166         tmpString.append("capacity = " + 512 + ", length = " + this.numInList + ", cursor = " + this.curr + "}");
167         return tmpString.toString();
168     }

```

12、getCurrentUsage

```

170     public Double getCurrentUsage() { 2个用法
171         return (double) numInList / msize;
172     }
173 }

```

(二) LList 类

```

1     package task1and2;
2
3     public class LList<T> implements List<T> { 0个用法
4         private Link head; 22个用法
5         private Link tail; 10个用法
6         private Link curr; 35个用法
7
8         public LList() { 0个用法
9             head = null;
10            curr = null;
11            tail = null;
12        }

```

1、insert

```

14      @Override 9个用法
15      public void insert(Object newElement) {
16          if (head == null) {
17              head = new Link(newElement, nextVal: null);
18              curr = head;
19              tail = head;
20              return;
21          }
22          Link temp = new Link(newElement, curr.next());
23          curr.setNext(temp);
24          if (curr == tail)
25              tail = temp;
26          curr = temp;
27      }
    
```

2、remove

```

29      @Override 6个用法
30      public void remove() {
31          if(head != null) {
32              System.out.println("the list is empty!");
33              return;
34          }
    
```

```

35          if (head == curr) {
36              head = head.next();
37              if (this.isEmpty()) {
38                  curr = tail = null;
39                  return;
40              }
41              curr = head;
42              return;
43          }
44          Link temp = head;
45          while (temp.next() != curr)
46              temp = temp.next();
47          temp.setNext(curr.next());
48          if (curr == tail) {
49              curr = head;
50              tail = temp;
51              return;
52          }
53          curr = curr.next();
54      }
    
```

3、replace 与 clear

```

56      @Override
57      public void replace(T newElement) {
58          if (curr == null) return;
59          curr.setElement(newElement);
60      }
61
62      @Override 6个用法
63      public void clear() { head = curr = tail = null; }
    
```

4、isEmpty 与 isFull

```

62      @Override 6个用法
63  > public void clear() { head = curr = tail = null; }
66
67      @Override
68  > public boolean isEmpty() { return head == null; }

```

5、四个 goto

```

77      @Override 6个用法
78  > public boolean gotoBeginning() {
79      |     if(isEmpty()) {
80      |         System.out.println("the list is empty!");
81      |         return false;
82      |     }
83      |     curr = head;
84      |     return true;
85      | }
86
87      @Override 6个用法
88  > public boolean gotoEnd() {
89      |     if(isEmpty()) {
90      |         System.out.println("the list is empty!");
91      |         return false;
92      |     } ;
93      |     curr = tail;
94      |     return true;
95      | }
96
97      @Override 6个用法
98  > public boolean gotoNext() {
99      |     if (this.isEmpty() || curr == tail) return false;
100     |     curr = curr.next();
101     |     return true;
102     | }
103
104     @Override 6个用法
105  > public boolean gotoPrev() {
106     |     if (this.isEmpty() || curr == head) return false;
107     |     Link temp = head;
108     |     while (temp.next() != curr)
109     |         temp = temp.next();
110     |     curr = temp;
111     |     return true;
112     | }

```

6、getCursor 与 currCount

```

114     @Override 0个用法
115  > public T getCursor() { return (T) curr.element(); }
118
119  v public int currCount() { 4个用法
120     |     int currNum = 0;
121     |     Link temp = head;
122     |     while (temp != curr) {
123     |         temp = temp.next();
124     |         currNum++;
125     |     }
126     |     return currNum;
127     | }

```

7、showStructure

```

129      @Override 7个用法
130      public void showStructure() {
131          if (this.isEmpty()) {
132              System.out.println("Empty list "+ "{capacity = " + S12 + ", length = " + cnt() + ", cursor = " + (currCount()-1) + "}"");
133              return;
134          }
135          Link temp = head;
136          while (temp != null) {
137              System.out.print(temp.element());
138              System.out.print(' ');
139              temp = temp.next();
140          }
141          System.out.println("{capacity = " + S12 + ", length = " + cnt() + ", cursor = " + currCount() + "}"");
142      }
    
```

8、moveToNth

```

144      @Override 0个用法
145      public void moveToNth(int n) throws ListException {
146          if (isEmpty() || n < 0 || n >= currCount()) {
147              System.out.println();
148          }
149          Link temp=head;
150          for(int i=0;i<n;i++){
151              temp=temp.next();
152          }
153          curr=temp;
154      }
    
```

9、getCurrentUsage

```

156      @Override 2个用法
157      public Double getCurrentUsage() {
158          return null;
159      }
    
```

10、find

```

161      @Override 0个用法
162      public boolean find(Object searchElement) throws ListException {
163          if(isEmpty()){
164              System.out.println("Invalid index");
165          }
166          Link initial=curr;
167          while(curr!=null){
168              curr=curr.next();
169              if(curr.element()==searchElement){
170                  return true;
171              }
172              if(curr.next()==null){
173                  return false;
174              }
175          }
176          return false;
177      }
    
```

11、cnt

```
178     public int cnt(){ 3个用法
179         int currNum = 0;
180         Link temp = head;
181         while (temp != null) {
182             temp = temp.next();
183             currNum++;
184         }
185         return currNum;
186     }
```

12、toString

```
189     public String toString() {
190         StringBuilder tmpString = new StringBuilder();
191         if (this.isEmpty()) {
192             tmpString = new StringBuilder("Empty list {capacity = 512, length = 0, cursor = -1}");
193             return tmpString.toString();
194         }
195         Link temp = head;
196         while (temp != null) {
197             tmpString.append(temp.element());
198             tmpString.append(' ');
199             temp = temp.next();
200         }
201         tmpString.append("{capacity = " + 512 + ", length = " + this.cnt() + ", cursor = " + this.currCount() + "}");
202         return tmpString.toString();
203     }
204 }
```

(三) DList 类

```
1     package task1and2;
2
3     public class DList<T> implements List<T> { 0个用法
4         private DLink head; 20个用法
5         private DLink tail; 11个用法
6         private DLink curr; 37个用法
7
8         public DList() { 0个用法
9             head = null;
10            curr = null;
11            tail = null;
12        }
```

1、Insert

```
14     @Override 9个用法
15     public void insert(Object newElement) {
16         if (head == null) {
17             head = new DLink(newElement, prevVal: null, nextVal: null);
18             curr = head;
19             tail = head;
20             return;
21         }
22         if (curr == tail) {
23             DLink temp = new DLink(newElement, curr, nextVal: null);
24             curr.setNext(temp);
25             tail = temp;
26             curr = temp;
27         } else {
28             DLink temp = new DLink(newElement, curr, curr.next());
29             curr.setNext(temp);
30             temp = temp.next();
31             temp.setPrev(curr.next());
32             curr = curr.next();
33         }
34     }
```

2、remove

```

36      @Override 6个用法
37      public void remove() {
38          assert head != null : "The list is empty!";

39          if (head == curr) {
40              head = head.next();
41              if (this.isEmpty()) {
42                  curr = tail = null;
43                  return;
44              }
45              head.setPrev(null);
46              curr = head;
47              return;
48          }
49          if (curr == tail) {
50              tail = curr.prev();
51              tail.setNext(null);
52              curr = head;
53          } else {
54              curr.next().setPrev(curr.prev());
55              curr.prev().setNext(curr.next());
56              curr = curr.next();
57          }
58      }

```

3、replace、clear、isEmpty、isFull

```

60      @Override
61      public void replace(Object newElement) {
62          if (curr == null) return;
63          curr.setElement(newElement);
64      }

65
66      @Override 6个用法
67      public void clear() { head = curr = tail = null; }

68
69
70
71      @Override
72      public boolean isEmpty() { return head == null; }

73
74
75
76      @Override 2个用法
77      public boolean isFull() { return false; }

```

4、四个 goto

```

81      @Override 6个用法
82      public boolean gotoBeginning() {
83          assert !this.isEmpty() : "the list is empty!";
84          curr = head;
85          return true;
86      }
87
88      @Override 6个用法
89      public boolean gotoEnd() {
90          assert !this.isEmpty() : "the list is empty!";
91          curr = tail;
92          return true;
93      }

```

```

95      @Override 6个用法
96      public boolean gotoNext() {
97          if (this.isEmpty() || curr == tail) return false;
98          curr = curr.next();
99          return true;
100     }
101
102     @Override 6个用法
103     public boolean gotoPrev() {
104         if (this.isEmpty() || curr == head) return false;
105         curr = curr.prev();
106         return true;
107     }

```

5、getCursor、currCount

```

109      @Override 0个用法
110      public T getCursor() { return (T) curr.element(); }
111
112
113
114      public int currCount() { 4个用法
115          int currNum = 0;
116          DLink temp = head;
117          while (temp != curr) {
118              temp = temp.next();
119              currNum++;
120          }
121          return currNum;
122      }

```

6、showStructure

```

124      @Override 7个用法
125      public void showStructure() {
126          if (this.isEmpty()) {
127              System.out.println("Empty list "+ "{capacity = " + 512 + ", length = " + cnt() + ", cursor = " + (currCount()-1) + "}");
128              return;
129          }
130          DLink temp = head;
131          while (temp != null) {
132              System.out.print(temp.element());
133              System.out.print(' ');
134              temp = temp.next();
135          }
136          int currNum = currCount();
137          System.out.println("{capacity = " + 512 + ", length = " + cnt() + ", cursor = " + currCount() + "}");
138      }
139

```

7、cnt

```

141      private int cnt() { 3个用法
142          int currNum = 0;
143          DLink temp = head;
144          while (temp != null) {
145              temp = temp.next();
146              currNum++;
147          }
148          return currNum;
149      }

```

8、moveToNth、getCurrentUsage、find

```

151     @Override 0 个用法
152     public void moveToNth(int n) throws ListException {
153
154     }
155
156     @Override 2 个用法
157     public Double getCurrentUsage() { return null; }
158
159
160
161     @Override 0 个用法
162     public boolean find(Object searchElement) throws ListException {
163         return false;
164     }

```

9、toString

```

166     public String toString() {
167         StringBuilder tmpString = new StringBuilder();
168         if (this.isEmpty()) {
169             tmpString = new StringBuilder("Empty list {capacity = 512, length = 0, cursor = -1}");
170             return tmpString.toString();
171         }
172         DLink temp = head;
173         while (temp != null) {
174             tmpString.append(temp.element());
175             tmpString.append(' ');
176             temp = temp.next();
177         }
178         tmpString.append("{capacity = " + 512 + ", length = " + this.cnt() + ", cursor = " + this.currCount() + "}");
179         return tmpString.toString();
180     }
181 }

```

(四) Link 类

```

1 package task1and2;
2
3 public class Link<T> { 20 个用法
4     private T element; 4 个用法
5     private Link next; 4 个用法
6
7     > public Link() { this( element: null, nextVal: null); }
8
9
10    > public Link(Link nextVal) { this( element: null, nextVal); }
11
12
13    public Link(T element, Link nextVal) { 4 个用法
14        this.element = element;
15        next = nextVal;
16    }
17
18
19
20    > public Link next() { return next; }
21
22
23    public Link setNext(Link next) { 2 个用法
24        this.next = next;
25        return this.next;
26    }
27
28
29    > public T element() { return element; }
30
31
32    public T setElement(T element) { 1 个用法
33        this.element = element;
34        return this.element;
35    }
36
37 }

```

(五) DLink 类

```

1 package task1and2;
2
3 public class DLink<T> { 24 个用法
4     private T element; 4 个用法
5     private DLink next; 4 个用法
6     private DLink prev; 4 个用法
7
8     > public DLink() { this( prevVal: null, nextVal: null); }
9
10
11
12 > public DLink(DLink prevVal, DLink nextVal) { this( element: null, prevVal, nextVal); }
13
14
15
16 public DLink(T element, DLink prevVal, DLink nextVal) { 4 个用法
17     this.element = element;
18     next = nextVal;
19     prev = prevVal;
20 }
21
22 > public DLink next() { return next; }
23
24
25
26 public DLink setNext(DLink next) { 4 个用法
27     this.next = next;
28     return this.next;
29 }
30
31 > public DLink prev() { return prev; }
32
33
34
35 public DLink setPrev(DLink prev) { 3 个用法
36     this.prev = prev;
37     return this.prev;
38 }
39
40 > public T element() { return element; }
41
42
43
44 public T setElement(T element) { 1 个用法
45     this.element = element;
46     return this.element;
47 }
48 }

```

(六) TestList 类（用于测试 LList 和 DList）

```

1 package task1and2;
2
3 import java.io.File;
4 import java.io.FileNotFoundException;
5 import java.io.PrintWriter;
6 import java.util.Scanner;
7
8 > public class TestList<T> {
9     public static void wrongLine(List wrongRecord, int line, String translatedList) throws FileNotFoundException, ListException {
10         Scanner fin = new Scanner(new File( pathname: "src\\task1and2\\list_result.txt"));
11         String tmp = new String();
12         for(int i = 0; i < line; i++){
13             tmp = fin.nextLine();
14         }
15         if(!tmp.equals(translatedList)){
16             wrongRecord.insert(line);
17         }
18     }
19     public static final int FREE_TEST = 1; 1 个用法
20     public static final int TEXT_TEST = 2; 2 个用法
21 }

```

```

100         case '\n':
101             list.clear();
102             continue;
103         default:
104             break;
105     }
106 }
107 list.showStructure();
108 writer.println(list.toString());
109 wrongLine(wrong, line, list.toString());
110 line++;
111 temp = fin.nextLine();
112 }
113 fin.close();
114 System.out.println("the followings are the wrong lines:");
115 System.out.println("(showed with list just defined, the last number is index of the wrong line list, ignore it)");
116 if (wrong.isEmpty()) {
117     System.out.println("Nothing Wrong");
118 } else {wrong.showStructure();}
119 }
120 }
121 }finally {
122     if (writer != null) {
123         writer.close();
124     }
125 }
126 }
127 }
128
129 public static void main(String[] args) throws FileNotFoundException, ListException {
130     List<Character> list = new DList<>();
131     translator(list, TEXT_TEST, outputFile: "src\\task1and2\\result.txt");
132 }
133 }

```

```

22 public static void translator(List<Character> list, int mode, String outputFile) throws FileNotFoundException, ListException {
23     PrintWriter writer = new PrintWriter(outputFile);
24     try {
25         if (mode == FREE_TEST) {
26             Scanner in = new Scanner(System.in);
27             String temp = in.nextLine();
28             while (!temp.equals("stop")) {
29                 for (int i = 0; i < temp.length(); i++) {
30                     switch (temp.charAt(i)) {
31                         case ' ':
32                             continue;
33                         case '+':
34                             list.insert(temp.charAt(i + 1));
35                             i++;
36                             continue;
37                         case '-':
38                             list.remove();
39                             continue;
40                         case '=':
41                             list.replace(temp.charAt(i + 1));
42                             i++;
43                             continue;
44                         case '#':
45                             list.gotoBeginning();
46                             continue;
47                         case '*':
48                             list.gotoEnd();
49                             continue;
50                         case '>':
51                             list.gotoNext();
52                             continue;
53                         case '<':
54                             list.gotoPrev();
55                             continue;
56                         case '\n':
57                             list.clear();
58                             continue;

```

```

59         default:
60             break;
61     }
62 }
63 list.showStructure();
64 temp = in.nextLine();
65 }
66 in.close();

```

```

67         } else if(mode == TEXT_TEST){
68             List wrong = new DList();
69             int line = 1;
70             Scanner fin = new Scanner(new File( pathname: "src\\task1and2\\list_testcase.txt"));
71             String temp = fin.nextLine();
72             while (!temp.equals("stop")) {
73                 for (int i = 0; i < temp.length(); i++) {
74                     switch (temp.charAt(i)) {
75                         case ' ':
76                             continue;
77                         case '+':
78                             list.insert(temp.charAt(i + 1));
79                             i++;
80                             continue;
81                         case '-':
82                             list.remove();
83                             continue;
84                         case '=':
85                             list.replace(temp.charAt(i + 1));
86                             i++;
87                             continue;
88                         case '#':
89                             list.gotoBeginning();
90                             continue;
91                         case '*':
92                             list.gotoEnd();
93                             continue;
94
95                         case '>':
96                             list.gotoNext();
97                             continue;
98                         case '<':
99                             list.gotoPrev();
100                             continue;
101
102                         case '~':
103                             list.clear();
104                             continue;
105                         default:
106                             break;
107                     }
108                 }
109                 list.showStructure();
110                 writer.println(list.toString());
111                 wrongLine(wrong, line, list.toString());
112                 line++;
113                 temp = fin.nextLine();
114             }
115             fin.close();
116             System.out.println("the followings are the wrong lines:");
117             System.out.println("(showed with list just defined, the last number is index of the wrong line list, ignore it)");
118             if (wrong.isEmpty()){
119                 System.out.println("Nothing Wrong");
120             }else {wrong.showStructure();}
121         }
122     }finally {
123         if (writer != null) {
124             writer.close();
125         }
126     }
127 }
128
129 public static void main(String[] args) throws FileNotFoundException, ListException {
130     List<Character> list = new DList<>();
131     translator(list, TEXT_TEST, outputFile: "src\\task1and2\\result.txt");
132 }
133 }

```

(七) TestListNew 类（用于测试 AList）

```

1 package task1and2;
2
3 import java.io.File;
4 import java.io.FileNotFoundException;
5 import java.util.Scanner;
6 import java.util.ArrayList;
7
8 public class TestListNew<T> {
9     public static void wrongLine(task1and2.List wrongRecord, int line, String translatedList) throws FileNotFoundException, ListException { 1个用法
10         Scanner fin = new Scanner(new File( pathname: "src\\task1and2\\list_testcase.txt"));
11         String tmp = new String();

```

```

12     for(int i = 0; i < line; i++){
13         tmp = fin.nextLine();
14     }
15     if(!tmp.equals(translatedList)){
16         wrongRecord.insert(line);
17     }
18 }
19 public static final int FREE_TEST = 1; 2个用法
20 public static final int TEXT_TEST = 2; 4个用法
21
22 public static void translator(taskland2.List<Character> list, int mode, ArrayList<Double> usageData) throws FileNotFoundException, ListException {
23     try{
24     }finally {
25         if (mode == FREE_TEST) {
26             Scanner in = new Scanner(System.in);
27             String temp = in.nextLine();
28             while (!temp.equals("stop")) {
29                 for (int i = 0; i < temp.length(); i++) {
30                     switch (temp.charAt(i)) {
31                         case ' ':
32                             continue;
33                         case '+':
34                             list.insert(temp.charAt(i + 1));
35                             i++;
36                             continue;
37                         case '-':
38                             list.remove();
39                             continue;
40                         case '=':
41                             list.replace(temp.charAt(i + 1));
42                             i++;
43                             continue;
44                         case '#':
45                             list.gotoBeginning();
46                             continue;

```

```

47                         case '*':
48                             list.gotoEnd();
49                             continue;
50                         case '>':
51                             list.gotoNext();
52                             continue;
53                         case '<':
54                             list.gotoPrev();
55                             continue;
56                         case '~':
57                             list.clear();
58                             continue;
59                         default:
60                             break;
61                     }
62                 }
63                 list.showStructure();
64                 temp = in.nextLine();
65             }
66             in.close();
67         } else if(mode == TEXT_TEST){
68             taskland2.List wrong = new ResizingAList();
69             int line = 1;
70             Scanner fin = new Scanner(new File("src\\taskland2\\list_testcase.txt"));
71             String temp = fin.nextLine();
72             while (!temp.equals("stop")) {
73                 for (int i = 0; i < temp.length(); i++) {
74                     switch (temp.charAt(i)) {
75                         case ' ':
76                             continue;
77                         case '+':
78                             list.insert(temp.charAt(i + 1));
79                             i++;
80                             continue;

```

```

81                         case '-':
82                             list.remove();
83                             continue;
84                         case '=':
85                             list.replace(temp.charAt(i + 1));
86                             i++;
87                             continue;
88                         case '#':
89                             list.gotoBeginning();
90                             continue;
91                         case '*':
92                             list.gotoEnd();
93                             continue;
94                         case '>':
95                             list.gotoNext();
96                             continue;
97                         case '<':
98                             list.gotoPrev();
99                             continue;
100                         case '~':
101                             list.clear();
102                             continue;

```

```

103         default:
104             break;
105     }
106 }
107 usageData.add(list.getCurrentUsage());
108 list.showStructure();
109 wrongLine(wrong, line, list.toString());
110 line++;
111 temp = fin.nextLine();
112 }
113 fin.close();
114
115 System.out.println("the followings are the wrong lines:");
116 System.out.println("(showed with list just defined, the last number is index of the wrong line list, ignore it)");
117 if (wrong.isEmpty()) {
118     System.out.println("Nothing Wrong");
119 } else { wrong.showStructure(); }
120
121 }
122 }
123
124 }

```

```

125 public static void translator2(taskland2.List<Character> list, int mode, ArrayList<Double> usageData) throws FileNotFoundException, ListException {
126     try {
127
128     } finally {
129         if (mode == FREE_TEST) {
130             Scanner in = new Scanner(System.in);
131             String temp = in.nextLine();
132             while (!temp.equals("stop")) {
133                 for (int i = 0; i < temp.length(); i++) {
134                     switch (temp.charAt(i)) {
135                         case ' ':
136                             continue;
137                         case '+':
138                             list.insert(temp.charAt(i + 1));
139                             i++;
140                             continue;
141                         case '-':
142                             list.remove();
143                             continue;
144                         case '=':
145                             list.replace(temp.charAt(i + 1));
146                             i++;
147                             continue;
148                         case '#':
149                             list.gotoBeginning();
150                             continue;
151                         case '*':
152                             list.gotoEnd();
153                             continue;
154                         case '>':
155                             list.gotoNext();
156                             continue;
157                         case '<':
158                             list.gotoPrev();
159                             continue;

```

```

160                         case '~':
161                             list.clear();
162                             continue;
163                         default:
164                             break;
165                     }
166                 }
167                 list.showStructure();
168                 temp = in.nextLine();
169             }
170             in.close();
171         } else if (mode == TEXT_TEST) {
172             Scanner fin = new Scanner(new File(pathname: "src\\taskland2\\list_testcase.txt"));
173             String temp = fin.nextLine();
174             while (!temp.equals("stop")) {
175                 for (int i = 0; i < temp.length(); i++) {
176                     switch (temp.charAt(i)) {
177                         case ' ':
178                             continue;
179                         case '+':
180                             list.insert(temp.charAt(i + 1));
181                             i++;
182                             continue;
183                         case '-':
184                             list.remove();
185                             continue;
186                         case '=':
187                             list.replace(temp.charAt(i + 1));
188                             i++;
189                             continue;
190                         case '#':
191                             list.gotoBeginning();
192                             continue;

```

```

193         case '*':
194             list.gotoEnd();
195             continue;
196         case '>':
197             list.gotoNext();
198             continue;
199         case '<':
200             list.gotoPrev();
201             continue;
202         case '~':
203             list.clear();
204             continue;
205         default:
206             break;
207     }
208 }
209 usageData.add((list).getCurrentUsage());
210 temp = fin.nextLine();
211 }
212 fin.close();
213
214 }
215
216 }
217
218 }
219
220 public static void main(String[] args) throws FileNotFoundException, ListException {
221     taskland2.List<Character> list = new ResizingAList<>();
222     List<Character> list1 = new AList<>();
223     ArrayList<Double>[] usageData = new ArrayList[] {new ArrayList<>(), new ArrayList<>()};
224     translator(list, TEXT_TEST, usageData[0]);
225     translator2(list1, TEXT_TEST, usageData[1]);
226 }
227 }

```

四、运行结果展示

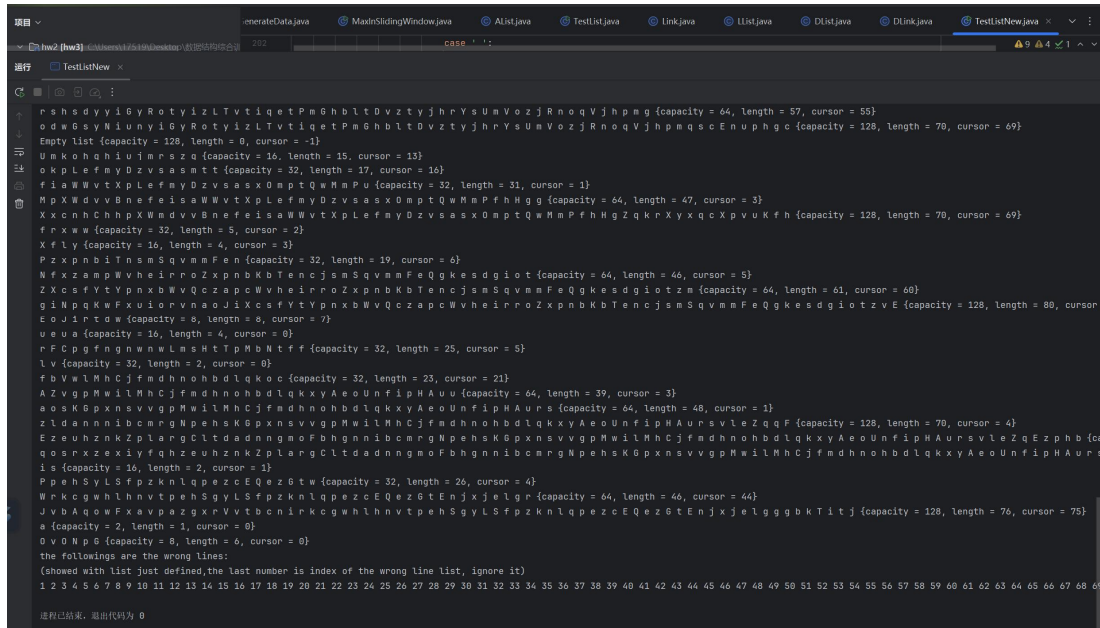
（一）AList 测试结果

```

运行 TestListNew
rshsdvyyigYRotyizLTvtigetPm6hbltDvztyjhrYsUmVozjRnoqVjhpmg {capacity = 64, length = 57, cursor = 55}
odw8syNiunyyigYRotyizLTvtigetPm6hbltDvztyjhrYsUmVozjRnoqVjhpmqscEnuphg {capacity = 128, length = 70, cursor = 69}
Empty list {capacity = 128, length = 0, cursor = -1}
Umkohqhiujmrszg {capacity = 16, length = 15, cursor = 13}
okpLefnyDzvsasmtt {capacity = 32, length = 17, cursor = 16}
fiawWvtXpLefnyDzvsasx0mptQwMmPu {capacity = 32, length = 31, cursor = 1}
MpxWdvvBnefeisaWwvtXpLefnyDzvsasx0mptQwMmPfhhg {capacity = 64, length = 47, cursor = 3}
XxcnhChhpXWmdvvBnefeisaWwvtXpLefnyDzvsasx0mptQwMmPfhhgZqkrXyxqcXpvuKfh {capacity = 128, length = 70, cursor = 69}
frxxw {capacity = 32, length = 5, cursor = 2}
Xfly {capacity = 16, length = 4, cursor = 3}
PzxpnbiTnsmSqvmmfen {capacity = 32, length = 19, cursor = 6}
MfxzaapWvheirrozxpnbKbTencjsmSqvmmfegkhesdgiot {capacity = 64, length = 46, cursor = 5}
ZXcsfYtYpnbWVqCzapcwvheirrozxpnbKbTencjsmSqvmmfegkhesdgiotzm {capacity = 64, length = 61, cursor = 60}
glnpqtwFkuioyvaaoJlXcsfYtYpnbWVqCzapcwvheirrozxpnbKbTencjsmSqvmmfegkhesdgiotzve {capacity = 128, length = 80, cursor = 75}
Eojirtdw {capacity = 6, length = 8, cursor = 7}
ueuaa {capacity = 16, length = 4, cursor = 0}
rfCpggfngnwWlmsHtIfpMbNtff {capacity = 32, length = 25, cursor = 5}
lv {capacity = 32, length = 2, cursor = 0}
fbvwlMhCjfmdbhnohbdlqkoc {capacity = 32, length = 23, cursor = 21}
AZvgpMwilmhCjfmdbhnohbdlqkxyAeoUnfipHAuu {capacity = 64, length = 39, cursor = 3}
aoaKGpXnsxvvgpMwilmhCjfmdbhnohbdlqkxyAeoUnfipHAurs {capacity = 64, length = 48, cursor = 1}
zldannnibcmrgNpehsKGpXnsxvvgpMwilmhCjfmdbhnohbdlqkxyAeoUnfipHAursvleZqqf {capacity = 128, length = 70, cursor = 4}
EzeuhznkZplargCltdadnngmoFbhgnnibcmrgNpehsKGpXnsxvvgpMwilmhCjfmdbhnohbdlqkxyAeoUnfipHAursvleZqEzphb {capacity = 128, length = 70, cursor = 4}
qoerxzexiyfqhzeuhznkZplargCltdadnngmoFbhgnnibcmrgNpehsKGpXnsxvvgpMwilmhCjfmdbhnohbdlqkxyAeoUnfipHAurs {capacity = 128, length = 70, cursor = 4}
is {capacity = 16, length = 2, cursor = 1}
PpehSvLSfpzknllqpezceQezGtw {capacity = 32, length = 26, cursor = 4}
WkcgahlnvtpesSgyLSfpzknllqpezceQezGtenjxjelgr {capacity = 64, length = 46, cursor = 44}
JvBAqgwFkavpazgxrvvtbcnirKcgwhlnvtpesSgyLSfpzknllqpezceQezGtenjxjelggbkTij {capacity = 128, length = 76, cursor = 75}
a {capacity = 2, length = 1, cursor = 0}
0v0NpG {capacity = 8, length = 6, cursor = 0}
the followings are the wrong lines:
(showed with list just defined, the last number is index of the wrong line list, ignore it)
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69

进程已结束，退出代码为 0

```



```
public class Node {
    char data;
    Node next;
}

public class List {
    private Node head;
    private int capacity;
    private int length;
    private int cursor;

    public List(int capacity) {
        this.capacity = capacity;
        this.length = 0;
        this.cursor = -1;
    }

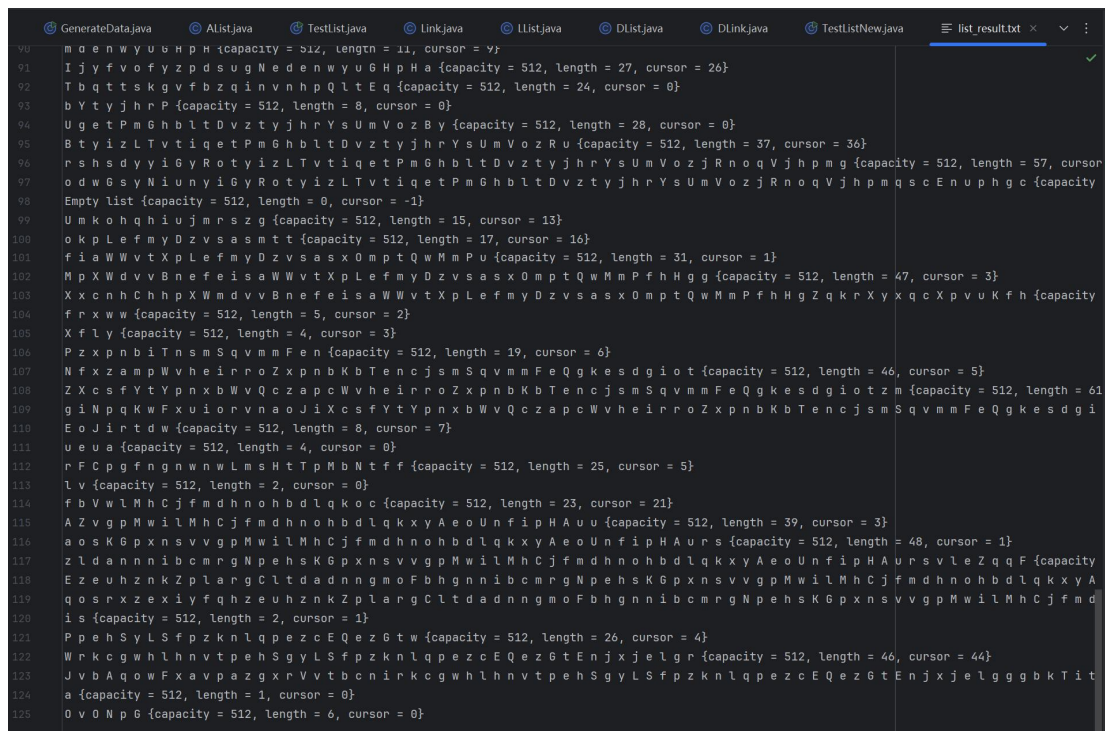
    public void add(char data) {
        if (length == capacity) {
            System.out.println("List is full, cannot add more elements.");
            return;
        }
        Node newNode = new Node();
        newNode.data = data;
        if (head == null) {
            head = newNode;
        } else {
            Node current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
        length++;
        cursor = length - 1;
    }

    public void delete(int index) {
        if (index < 0 || index >= length - 1) {
            System.out.println("Index out of range, cannot delete element at index " + index);
            return;
        }
        if (index == 0) {
            head = head.next;
        } else {
            Node current = head;
            for (int i = 0; i < index; i++) {
                current = current.next;
            }
            current.next = current.next.next;
        }
        length--;
        cursor = length - 1;
    }

    public void traverse() {
        if (length == 0) {
            System.out.println("List is empty.");
            return;
        }
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }
}
```

AList 实现成功。

(二) LList 测试结果



```
public class Node {
    char data;
    Node next;
}

public class List {
    private Node head;
    private int capacity;
    private int length;
    private int cursor;

    public List(int capacity) {
        this.capacity = capacity;
        this.length = 0;
        this.cursor = -1;
    }

    public void add(char data) {
        if (length == capacity) {
            System.out.println("List is full, cannot add more elements.");
            return;
        }
        Node newNode = new Node();
        newNode.data = data;
        if (head == null) {
            head = newNode;
        } else {
            Node current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
        length++;
        cursor = length - 1;
    }

    public void delete(int index) {
        if (index < 0 || index >= length - 1) {
            System.out.println("Index out of range, cannot delete element at index " + index);
            return;
        }
        if (index == 0) {
            head = head.next;
        } else {
            Node current = head;
            for (int i = 0; i < index; i++) {
                current = current.next;
            }
            current.next = current.next.next;
        }
        length--;
        cursor = length - 1;
    }

    public void traverse() {
        if (length == 0) {
            System.out.println("List is empty.");
            return;
        }
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }
}
```

LList 实现成功。

(三) DList 测试结果

```
运行 Testlist <
rshsdyyiGyRotyizLTvtiqetPmGhbLTDvztYjhrYsUmVozJRnoqVjhpmg {capacity = 512, length = 57, cursor = 55}
odw6syNlunyigYRotyizLTvtiqetPmGhbLTDvztYjhrYsUmVozJRnoqVjhpmsg {capacity = 512, length = 70, cursor = 69}
Empty list {capacity = 512, length = 0, cursor = -1}
UmkohqhIujmrszg {capacity = 512, length = 15, cursor = 13}
okpLefmyDzvsasmtt {capacity = 512, length = 17, cursor = 16}
fiaWWvtXpLefmyDzvsasx0mptQwMmPu {capacity = 512, length = 31, cursor = 1}
MpXWdvVBnefeisaWWvtXpLefmyDzvsasx0mptQwMmPfHgg {capacity = 512, length = 47, cursor = 3}
XxcnChhXWdvVBnefeisaWWvtXpLefmyDzvsasx0mptQwMmPfHggZqkRxyxqcXpvUkfh {capacity = 512, length = 70, cursor = 69}
frxww {capacity = 512, length = 5, cursor = 2}
XfLy {capacity = 512, length = 4, cursor = 3}
PzxpnbiTnsMSqvmmfEn {capacity = 512, length = 19, cursor = 6}
NfxzampWvheirrozXpnBKbTencjmsSqvmmfEQgkesdgiot {capacity = 512, length = 46, cursor = 5}
ZXcsfYtYpnxbWwQczapcWvheirrozXpnBKbTencjmsSqvmmfEQgkesdgiotzm {capacity = 512, length = 61, cursor = 60}
giNpqKWfxuioRVnaoJiXcsfYtYpnxbWwQczapcWvheirrozXpnBKbTencjmsSqvmmfEQgkesdgiotzve {capacity = 512, length = 80, cursor = 69}
EoJirtw {capacity = 512, length = 8, cursor = 7}
ueua {capacity = 512, length = 4, cursor = 0}
rFCpgfgngnnwLmsHtTpMbNtff {capacity = 512, length = 25, cursor = 5}
lv {capacity = 512, length = 2, cursor = 0}
fbVwLMhCjfmDhnohbdlqkoc {capacity = 512, length = 23, cursor = 21}
AZvgpMwilmhCjfmDhnohbdlqkxyAeoUnfiPHAu {capacity = 512, length = 39, cursor = 3}
aOSK6pxnsvvgpMwilmhCjfmDhnohbdlqkxyAeoUnfiPHAu {capacity = 512, length = 48, cursor = 1}
zldannnibcmrgNpehsK6pxnsvvgpMwilmhCjfmDhnohbdlqkxyAeoUnfiPHAu {capacity = 512, length = 70, cursor = 4}
EzeuhznkZplargCLtdadnngmoFbhgnnibcmrgNpehsK6pxnsvvgpMwilmhCjfmDhnohbdlqkxyAeoUnfiPHAu {capacity = 512, length = 70, cursor = 4}
qosrxzexiyfghzeuhznkZplargCLtdadnngmoFbhgnnibcmrgNpehsK6pxnsvvgpMwilmhCjfmDhnohbdlqkxyAeoUnfiPHAu {capacity = 512, length = 2, cursor = 1}
PpehSyLSfpzknLqpezceQEzGtw {capacity = 512, length = 26, cursor = 4}
WrKcgwhlhnvtpehSgyLSfpzknLqpezceQEzGtEnjxjelgr {capacity = 512, length = 46, cursor = 44}
JvbAqowFxaVPazgxrvvtbcnirKcgwhlhnvtpehSgyLSfpzknLqpezceQEzGtEnjxjelgggbkTitj {capacity = 512, length = 76, cursor = 75}
a {capacity = 512, length = 1, cursor = 0}
OvONpG {capacity = 512, length = 6, cursor = 0}
the followings are the wrong lines:
(showed with list just defined,the last number is index of the wrong line list, ignore it)
Nothing Wrong

进程已结束，退出代码为 0
```

```
a GenerateData.java AList.java Testlist.java Link.java LList.java DList.java DLink.java TestListNew.java list_result.txt
90 m o e n w y u o h p h {capacity = 512, length = 11, cursor = 1}
91 I j y v o f y z p d s u g N e d e n w y u G h p H a {capacity = 512, length = 27, cursor = 26}
92 T b q t t s k g v f b z q i n v n h p Q l t E q {capacity = 512, length = 24, cursor = 0}
93 b Y t y j h r P {capacity = 512, length = 8, cursor = 0}
94 U g e t P m G h b l t D v z t y j h r Y s U m V o z B y {capacity = 512, length = 28, cursor = 0}
95 B t y i z L T v t i q e t P m G h b l t D v z t y j h r Y s U m V o z R u {capacity = 512, length = 37, cursor = 36}
96 r s h s d y y i G y R o t y i z L T v t i q e t P m G h b l t D v z t y j h r Y s U m V o z j R n o q V j h p m g {capacity = 512, length = 57, cursor = 55}
97 o d w 6 s y N i u n y i g Y R o t y i z L T v t i q e t P m G h b l t D v z t y j h r Y s U m V o z j R n o q V j h p m g s c E n u p h g c {capacity = 512, length = 70, cursor = 69}
98 Empty list {capacity = 512, length = 0, cursor = -1}
99 U m k o h q h i u j m r s z g {capacity = 512, length = 15, cursor = 13}
100 o k p L e f m y D z v s a s m t t {capacity = 512, length = 17, cursor = 16}
101 f i a W W v t X p L e f m y D z v s a s x 0 m p t Q w M m P u {capacity = 512, length = 31, cursor = 1}
102 M p X W d v v B n e f e i s a W W v t X p L e f m y D z v s a s x 0 m p t Q w M m P f h H g g {capacity = 512, length = 47, cursor = 3}
103 X x c n h C h h p X W m d v v B n e f e i s a W W v t X p L e f m y D z v s a s x 0 m p t Q w M m P f h H g g Z q k R X y x q c X p v u K f h {capacity = 512, length = 70, cursor = 69}
104 f r x w w {capacity = 512, length = 5, cursor = 2}
105 X f l y {capacity = 512, length = 4, cursor = 3}
106 P z x p n b i T n s m S q v m m f e n {capacity = 512, length = 19, cursor = 6}
107 N f x z a m p W v h e i r r o z X p n b K b T e n c j m s S q v m m f e Q g k e s d g i o t {capacity = 512, length = 46, cursor = 5}
108 Z X c s f Y t Y p n x b W w Q c z a p c W v h e i r r o z X p n b K b T e n c j m s S q v m m f e Q g k e s d g i o t z m {capacity = 512, length = 61, cursor = 60}
109 g i N p q K W F x u i o r v n a o J i X c s f Y t Y p n x b W w Q c z a p c W v h e i r r o z X p n b K b T e n c j m s S q v m m f e Q g k e s d g i o t z v e {capacity = 512, length = 80, cursor = 69}
110 E o J i r t w {capacity = 512, length = 8, cursor = 7}
111 u e u a {capacity = 512, length = 4, cursor = 0}
112 r F C p g f g n g n n w L m s H t T p M b N t f f {capacity = 512, length = 25, cursor = 5}
113 l v {capacity = 512, length = 2, cursor = 0}
114 f b V w L M h C j f m D h n o h b d l q k o c {capacity = 512, length = 23, cursor = 21}
115 A Z v g p M w i l M h C j f m D h n o h b d l q k x y A e o U n f i P H A u {capacity = 512, length = 39, cursor = 3}
116 a o s K 6 p x n s v v g p M w i l M h C j f m D h n o h b d l q k x y A e o U n f i P H A u r s {capacity = 512, length = 48, cursor = 1}
117 z l d a n n n i b c m r g N p e h s K 6 p x n s v v g p M w i l M h C j f m D h n o h b d l q k x y A e o U n f i P H A u r s v l e Z q q F {capacity = 512, length = 70, cursor = 4}
118 E z e u h z n k Z p l a r g C l t d a d n n g m o F b h g n n i b c m r g N p e h s K 6 p x n s v v g p M w i l M h C j f m D h n o h b d l q k x y A e o U n f i P H A u r s v l e Z q q F {capacity = 512, length = 70, cursor = 4}
119 q o s r x z e x i y f g h z e u h z n k Z p l a r g C l t d a d n n g m o F b h g n n i b c m r g N p e h s K 6 p x n s v v g p M w i l M h C j f m D h n o h b d l q k x y A e o U n f i P H A u r s v l e Z q q F {capacity = 512, length = 2, cursor = 1}
120 i s {capacity = 512, length = 2, cursor = 1}
121 P p e h S y L S f p z k n L q p e z c e Q e z G t w {capacity = 512, length = 26, cursor = 4}
122 W r K c g w h l h n v t p e h S g y L S f p z k n L q p e z c e Q e z G t E n j x j e l g r {capacity = 512, length = 46, cursor = 44}
123 J v b A q o w F x a v p a z g x r v v t b c n i r K c g w h l h n v t p e h S g y L S f p z k n L q p e z c e Q e z G t E n j x j e l g g g b k T i t j {capacity = 512, length = 76, cursor = 75}
124 a {capacity = 512, length = 1, cursor = 0}
125 O v O N p G {capacity = 512, length = 6, cursor = 0}
```

DList 实现成功。

(四) 时间复杂度表

	AList	LList	DList
insert()	$O(n)$	$O(1)$	$O(1)$
remove()	$O(n)$	$O(n)$	$O(1)$
replace()	$O(1)$	$O(1)$	$O(1)$
clear()	$O(1)$	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$	$O(1)$
isFull()	$O(1)$	$O(1)$	$O(1)$
gotoBeginning()	$O(1)$	$O(1)$	$O(1)$
gotoEnd()	$O(1)$	$O(1)$	$O(1)$
gotoNext()	$O(1)$	$O(1)$	$O(1)$
gotoPrev()	$O(1)$	$O(n)$	$O(1)$
getCursor()	$O(1)$	$O(1)$	$O(1)$
showStructure()	$O(n)$	$O(n)$	$O(n)$

各个操作的时间复杂度表

五、总结与收获

在完成任务 1 的过程中，我深入学习了如何为 List ADT 实现不同的数据结构，包括顺序数组、单向链表和双向链表。通过这个实验，我对各种数据结构的内部工作原理有了更深入的理解，尤其是它们在处理 Character 类型数据时的表现。

实验的一个重要部分是使用测试用例来验证数据结构的正确性。这不仅帮助我确保代码的功能正确无误，而且还锻炼了我分析和调试代码的能力。通过对 showStructure 方法的不断调用，我能够直观地看到每个命令的执行结果和列表的状态变化。

此外，我还学习了如何为每种实现计算和列举时间复杂度。这一部分对于理解不同数据结构的效率至关重要，也帮助我在未来的项目中做出更合理的数据结构选择。

这次实验不仅提高了我的编程技能，加深了我对数据结构理论知识的理解。

任务 2：为指定的 DQueue ADT 实现 各种数据结构

一、题目

在本次实验中，完成双端队列抽象数据类型的实现。双端队列是普通队列的一个扩展，其允许在队头和队尾执行入队和出队操作，因此双端队列兼具了栈和队列的操作能力。

1、为指定的 DQueue ADT（该 ADT 的具体要求请参见文件 DQueue.java）实现两种数据结构：

①使用顺序数组做为存储表示； ②使用双向链表做为存储表示。不论使用哪种存储表示实现的数据结构，实验中需要处理的数据类型都是 Integer 类型，且对 ADT 中的每个行为要求在最坏情形下的时间复杂度都必须满足 $O(1)$ 。

2、借助合适的数据结构解决如下问题。

问题描述：给定一个包含 n ($n > 0$) 个数据的序列，和一个 K ($0 < K \leq n$) 值，要求计算序列中每 K 个连续元素构成的子序列的最大值。

资源要求：最坏情形下的时间复杂度为 $O(n)$ 。

举例：当数据序列为：8、5、10、7、9、4、15、12、90、13 和 $K=4$ ，那么每 K 个连续的子序列的最大值分别为：10、10、10、15、15、90 和 90。

随机生成 10 组数据完成测试。

二、算法设计

在本次实验中，我们设计并实现了双端队列（DQueue）抽象数据类型，并利用其解决了滑动窗口最大值的问题。

我们提供了顺序数组和双向链表两种数据结构的实现。顺序数组实现的双端队列通过维护前后指针来高效地进行元素的添加和删除，确保在最坏情况下，所有操作的时间复杂度均为 $O(1)$ 。双向链表通过链表节点的前驱和后继指针，来实现在任意一端进行操作。

在解决滑动窗口最大值问题时，我们使用了双端队列来存储当前窗口内元素的索引。通过维护一个单调递减的队列，我们能够高效地获取当前窗口的最大值。每当窗口滑动时，我们只需检查和更新队列，确保队列的头部始终是当前窗口的最大值，从而将整个过程的时间复杂度控制在 $O(n)$ 。

三、主干代码说明

（一）顺序数组

```
1 package task1and2;
2
3 public class ArrayDQueue<T> implements DQueue<T> { 0 个用法
4     private T[] elements; 7 个用法
5     private int front, rear, size, capacity; 12 个用法
6
7     @SuppressWarnings("unchecked") 0 个用法
8     public ArrayDQueue(int capacity) {
9         this.capacity = capacity;
10        this.elements = (T[]) new Object[capacity];
11        this.front = -1;
12        this.rear = 0;
13        this.size = 0;
14    }
```

1、isEmpty

```
16  public boolean isEmpty() {
17     |     return size == 0;
18     }
```

2、isFull

```
20  public boolean isFull() { 2个用法
21     |     return size == capacity;
22     }
```

3、size

```
24  public int size() {
25     |     return size;
26     }
```

4、enqueueToRear

```
28  public void enqueueToRear(T element) throws ListException { 0个用法
29     |     if (isFull()) throw new ListException("Queue is full");
30     |     rear = (rear + 1) % capacity;
31     |     elements[rear] = element;
32     |     if (front == -1) front = rear; // First element added
33     |     size++;
34     }
```

5、enqueueToFront

```
36  public void enqueueToFront(T element) throws ListException { 0个用法
37     |     if (isFull()) throw new ListException("Queue is full");
38     |     front = (front - 1 + capacity) % capacity;
39     |     elements[front] = element;
40     |     if (rear == -1) rear = front; // First element added
41     |     size++;
42     }
```

6、dequeueFromFront

```
44  public T dequeueFromFront() { 0个用法
45     |     if (isEmpty()) return null;
46     |     T result = elements[front];
47     |     front = (front + 1) % capacity;
48     |     size--;
49     |     if (size == 0) front = -1; // Reset if empty
50     |     return result;
51     }
```

7、dequeueFromRear

```
53  public T dequeueFromRear() { 0个用法
54     |     if (isEmpty()) return null;
55     |     T result = elements[rear];
56     |     rear = (rear - 1 + capacity) % capacity;
57     |     size--;
58     |     if (size == 0) rear = -1; // Reset if empty
59     |     return result;
60     }
```

8、getFront

```
62 ① ↑  public T getFront() { 0个用法
63      |     return isEmpty() ? null : elements[front];
64      | }

```

9、getRear

```
66 ① ↑  public T getRear() { 0个用法
67      |     return isEmpty() ? null : elements[rear];
68      | }
69      | }

```

(二) 链式存储实现线性表

```
1  package taskland2;
2
3  class Node<T> { 7个用法
4      |     T data; 5个用法
5      |     Node<T> next; 4个用法
6      |     Node<T> prev; 4个用法
7
8      |     Node(T data) { 2个用法
9      |     |     this.data = data;
10     |     }
11     | }
12
13     public class LinkedListDQueue<T> implements DQueue<T> { 0个用法
14         |     private Node<T> front, rear; 13个用法
15         |     private int size; 7个用法
16
17         |     public LinkedListDQueue() { 0个用法
18         |     |     front = rear = null;
19         |     |     size = 0;
20         |     }

```

1、isEmpty

```
22 ① ↑  public boolean isEmpty() {
23      |     return size == 0;
24      | }

```

2、isFull

```
26 ① ↑  public boolean isFull() { 2个用法
27      |     return false; // 链表实现不可能满
28      | }

```

3、size

```
30 ① ↑  public int size() { 0个用法
31      |     return size;
32      | }

```

4、enqueueToRear

```

34 ④  public void enqueueToRear(T element) throws ListException { 0个用法
35      if (element == null) throw new ListException("Element cannot be null");
36      Node<T> newNode = new Node<>(element);
37      if (isEmpty()) {
38          front = rear = newNode;
39      } else {
40          rear.next = newNode;
41          newNode.prev = rear;
42          rear = newNode;
43      }
44      size++;
45  }

```

5、enqueueToFront

```

47 ④  public void enqueueToFront(T element) throws ListException { 0个用法
48      if (element == null) throw new ListException("Element cannot be null");
49      Node<T> newNode = new Node<>(element);
50      if (isEmpty()) {
51          front = rear = newNode;
52      } else {
53          newNode.next = front;
54          front.prev = newNode;
55          front = newNode;
56      }
57      size++;
58  }

```

6、dequeueFromFront

```

60 ④  public T dequeueFromFront() { 0个用法
61      if (isEmpty()) return null;
62      T result = front.data;
63      front = front.next;
64      if (front != null) front.prev = null;
65      else rear = null; // Reset rear if empty
66      size--;
67      return result;
68  }

```

7、dequeueFromRear

```

70 ④  public T dequeueFromRear() { 0个用法
71      if (isEmpty()) return null;
72      T result = rear.data;
73      rear = rear.prev;
74      if (rear != null) rear.next = null;
75      else front = null; // Reset front if empty
76      size--;
77      return result;
78  }

```

8、getFront

```

80 ④  public T getFront() { 0个用法
81      return isEmpty() ? null : front.data;
82  }

```

9、getRear

```

84  public T getRear() { 0个用法
85      return isEmpty() ? null : rear.data;
86  }
87  }

```

(三) 子序列最大值

1、maxSlidingWindow 方法

```

1  package taskland2;
2  import java.util.ArrayList;
3  import java.util.List;
4
5  public class MaxInSlidingWindow {
6  @ public static List<Integer> maxSlidingWindow(int[] nums, int k) { 1个用法
7      List<Integer> result = new ArrayList<>();
8      DQueue<Integer> deque = new LinkedListDQueue<>();
9
10     for (int i = 0; i < nums.length; i++) {
11         // 移除不在当前窗口中的元素
12         while (!deque.isEmpty() && deque.getFront() < i - k + 1) {
13             deque.dequeueFromFront();
14         }
15         // 移除所有小于当前元素的元素
16         while (!deque.isEmpty() && nums[deque.getRear()] < nums[i]) {
17             deque.dequeueFromRear();
18         }
19         // 在双端队列的尾部添加当前元素的索引
20         try {
21             deque.enqueueToRear(i);
22         } catch (ListException e) {
23             e.printStackTrace(); // 处理异常
24         }
25         // 当第一个窗口完成时，添加最大值到结果中
26         if (i >= k - 1) {
27             result.add(nums[deque.getFront()]);
28         }
29     }
30
31     return result;
32 }

```

2、主函数

```

35  public static void main(String[] args) {
36      Random random = new Random();
37      int numTests = 10; // 生成10组测试数据
38      int arraySize = 10; // 每组数据的大小
39
40      for (int test = 0; test < numTests; test++) {
41          // 随机生成一组数据
42          int[] data = new int[arraySize];
43          for (int i = 0; i < arraySize; i++) {
44              data[i] = random.nextInt(100); // 随机生成0到99之间的整数
45          }
46          int k = random.nextInt(arraySize) + 1; // 随机生成1到arraySize之间的k值
47
48          // 打印生成的数据和k值
49          System.out.println("测试数据: " + java.util.Arrays.toString(data) + ", k = " + k);
50
51          // 计算每个窗口的最大值
52          List<Integer> maxValues = maxSlidingWindow(data, k);
53          System.out.println("滑动窗口最大值: " + maxValues);
54          System.out.println(); // 添加空行以便于输出阅读
55      }
56  }
57

```

四、运行结果展示

```
D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2.4\lib\idea_rt.jar=6037
测试数据: [14, 96, 46, 58, 99, 70, 1, 42, 43, 73], k = 2
滑动窗口最大值: [96, 96, 58, 99, 99, 70, 42, 43, 73]

测试数据: [63, 42, 48, 64, 93, 76, 55, 18, 78, 42], k = 8
滑动窗口最大值: [93, 93, 93]

测试数据: [32, 15, 42, 52, 94, 22, 88, 91, 42, 23], k = 2
滑动窗口最大值: [32, 42, 52, 94, 94, 88, 91, 91, 42]

测试数据: [31, 13, 67, 21, 1, 29, 94, 93, 64, 35], k = 9
滑动窗口最大值: [94, 94]

测试数据: [44, 76, 74, 81, 91, 46, 22, 99, 45, 10], k = 7
滑动窗口最大值: [91, 99, 99, 99]

测试数据: [1, 40, 92, 75, 21, 38, 16, 92, 30, 84], k = 3
滑动窗口最大值: [92, 92, 92, 75, 38, 92, 92, 92]

测试数据: [15, 33, 65, 29, 97, 86, 92, 99, 14, 80], k = 8
滑动窗口最大值: [99, 99, 99]

测试数据: [6, 16, 89, 80, 66, 43, 7, 30, 40, 93], k = 2
滑动窗口最大值: [16, 89, 89, 80, 66, 43, 30, 40, 93]

测试数据: [69, 47, 22, 50, 35, 49, 16, 25, 59, 76], k = 9
滑动窗口最大值: [69, 76]

测试数据: [84, 4, 92, 33, 31, 20, 26, 82, 66, 55], k = 4
滑动窗口最大值: [92, 92, 92, 33, 82, 82, 82]

进程已结束, 退出代码为 0
```

子序列最大值问题实现成功。

五、总结与收获

通过本次实验,我加深了对数据结构和算法设计的理解,尤其是双端队列的实现及其应用。首先,我认识到不同数据结构在特定场景下的优势和适用性。例如,顺序数组实现适合空间固定且访问模式简单的情况,而双向链表则更适合动态变化的场景。这让我意识到在解决实际问题时,选择合适的数据结构是非常重要的。

其次,利用双端队列解决滑动窗口最大值的问题让我体验到了算法优化的重要性。在这个任务中,通过合理设计队列的操作,我能够在 $O(n)$ 的时间复杂度内完成任务,这大大提升了效率。

任务 3：栈

子任务一：

一、题目：非递归快速排序

众所周知，栈虽然是一个操作受限的线性表，但是其用途却很广泛，栈的数据结构实现非常简单，因此我们只从应用层面熟悉栈。请完成下面三个子任务。

①递归是一种解决很多复杂问题最简单的思想方法，而任何编程语言对递归程序的支持都是通过栈实现的。请利用课堂上讲解的“Hanoi 塔”问题的非递归转化方法完成对递归快速排序的非递归转化。

二、数据结构设计

数据结构是一个可比较元素的数组 `Comparable[] objs`。这种结构允许算法处理各种类型的数据，只要这些数据可以进行比较操作。

本题程序用一个内部类 `Range` 来记录当前待排序的序列的起始和终止位置。在每一次迭代中，从堆栈中 `pop` 出一个 `Range` 对象，这个 `Range` 对象表示当前待处理和待排序的序列。

三、算法设计

本实验的核心是将递归快速排序算法转化为非递归形式。

每经过一次遍历，就将 `Range` 中的序列分为两个部分，其中一部分的所有元素都不大于另一部分的所有元素，并得到分区点 `pivot`。[`pivot - 1, low`] 与 [`pivot + 1, high`] 这两个子区间将会被创建为新的 `Range` 对象，然后被 `push` 到堆栈中。堆栈保证了最小的子区间首先被处理，这样可以尽快地释放空间。

这就说明可以通过使用栈（`Stack<Range>`）来模拟递归调用栈的方式实现。栈用于存储待排序的数组范围，模拟了递归过程中的调用和返回。

四、主干代码说明

（一）QuickSortOfRecursion

```
1 package task3of1;
2
3 import java.util.Stack;
4
5 public class QuickSortOfRecursion extends SortAlgorithm {
6     @Override 3 个用法
7     public void sort(Comparable[] objs) {
8         Stack<Range> stack=new Stack<>();
9         stack.push(new Range( low: 0, high: objs.length-1));
10
11         while (!stack.isEmpty()) {
12             Range range=stack.pop();
13             if(range.low<range.high){
14                 int pivot=partition(objs, range.low,range.high);
15                 stack.push(new Range(range.low, high: pivot-1));
16                 stack.push(new Range( low: pivot+1,range.high));
17             }
18         }
19     }
```

```
21  @ private int partition(Comparable[] objs,int low,int high){ 1个用法
22      Comparable pivot=objs[high];
23      int i=low-1;
24      for (int j=low;j<high;j++){
25          if(!less(objs[j],pivot)||equals(objs[j],pivot)){
26              i++;
27              exchange(objs,i,j);
28          }
29      }
30      exchange(objs,i+1,high);
31      return i+1;
32  }
33
34  static class Range{ 5个用法
35      int low,high; 4个用法
36      public Range(int low,int high){ 3个用法
37          this.high=high;
38          this.low=low;
39      }
40  }
```

(二) 测试程序

```
42  public static void main(String[] args) {
43      QuickSortOfRecursion sorter = new QuickSortOfRecursion();
44      Comparable[] arr = {1, 5, 7, 0, -1, 40, 10, 7, -8, 3, 9, 56, 10, 2, 11, 97, 7, 2};
45      System.out.println("The original array: ");
46      System.out.println("[1, 5, 7, 0, -1, 40, 10, 7, -8, 3, 9, 56, 10, 2, 11, 97, 7, 2]");
47      sorter.sort(arr);
48      System.out.println("Sorted array: ");
49      sorter.show(arr);
50  }
51
52 }
```

五、运行结果展示

```
D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2.4
The original array:
[1, 5, 7, 0, -1, 40, 10, 7, -8, 3, 9, 56, 10, 2, 11, 97, 7, 2]
Sorted array:
-8 -1 0 1 2 2 3 5 7 7 7 9 10 10 11 40 56 97

进程已结束，退出代码为 0
```

说明非递归的快速排序算法完全正确

六、总结与收获

通过对子任务一的学习，我更深入地理解了快速排序的工作原理和其非递归版本的实现，同时也学习到了如何使用栈数据结构解决问题的技巧。

这个例子也证明了，通常，递归算法总是可以转换为使用栈的非递归算法，虽然在大多数情况下，递归方法更简洁，但是当递归深度非常大时，使用非递归的方法可以避免栈溢出。对于快速排序算法，非递归形式在整个排序过程中，通过交换元素的方式在原地进行排序，这也节省了额外的存储空间。

在这个过程中，我也加深了对快速排序算法理论的理解，也锻炼了自己的编程能力。

子任务二：

一、题目：混合运算表达式

②算术混合运算表达式的计算。表达式不仅能处理整数，还需要处理小数。表达式中涉及的运算符包括+、-、*、/、^(指数)。表达式可以包含括号（只包含圆括号）嵌套，因此要处理括号匹配失败的情形。

二、数据结构设计

本实验的核心使用两个栈来存储和处理表达式：

使用 `number` 栈用于存储操作数，`operator` 栈用于存储操作符。

通过遍历表达式字符串，识别数字和操作符，并按照运算规则进行入栈和出栈操作，当遇到右括号时，计算栈内的所有操作，直到遇到左括号为止。最终将栈内剩余的操作数和操作符依次计算，得到最终结果。

三、主干代码说明

（一）测试程序

```
1 package task3of2;
2
3 import java.util.Stack;
4
5 public class ExpressionCalculator {
6     public static double calculator(String expression) { 1个用法
7         Stack<Double> number = new Stack<>();
8         Stack<Character> operator = new Stack<>();
9         for (int i = 0; i < expression.length(); i++) {
10             char c = expression.charAt(i);
11             if (c == ' ') {
12                 continue;
13             }
14
15             if (Character.isDigit(c) || c == '.') {
16                 StringBuilder sb = new StringBuilder();
17                 while (i < expression.length() && (Character.isDigit(expression.charAt(i)) || expression.charAt(i) == '.')) {
18                     sb.append(expression.charAt(i++));
19                 }
20                 i--;
21                 number.push(Double.parseDouble(sb.toString()));
22             } else if (c == '(') {
23                 operator.push(c);
24
25             } else if (c == ')') {
26                 while (!operator.isEmpty() && operator.peek() != '(') {
27                     number.push(applyOp(operator.pop(), number.pop(), number.pop()));
28                 }
29                 if (!operator.isEmpty()) {
30                     operator.pop(); // Pop the '('
31                 }
32             } else if ("+-*/^".indexOf(c) >= 0) {
33                 while (!operator.isEmpty() && hasPrecedence(c, operator.peek())) {
34                     number.push(applyOp(operator.pop(), number.pop(), number.pop()));
35                 }
36                 operator.push(c);
37             }
38         }
39
40         while (!operator.isEmpty()) {
41             number.push(applyOp(operator.pop(), number.pop(), number.pop()));
42         }
43         return number.pop();
44     }
45 }
```

```

44     private static boolean hasPrecedence(char c, Character peek) { 1个用法
45         if (peek == '(' || peek == ')') {
46             return false;
47         }
48         if ((c == '*' || c == '/') && (peek == '+' || peek == '-')) {
49             return false;
50         }
51         return !(c == '^' && (peek == '+' || peek == '-' || peek == '*' || peek == '/'));
52     }
53
54     private static double applyOp(char op, double num2, double num1) { 3个用法
55         switch (op) {
56             case '+':
57                 return num1 + num2;
58             case '-':
59                 return num1 - num2;
60             case '*':
61                 return num1 * num2;
62             case '/':
63                 if (num2 == 0) throw new ArithmeticException("Cannot divide by zero");
64                 return num1 / num2;
65             case '^':
66                 return Math.pow(num1, num2);
67         }
68         return 0;
69     }

```

(二) 测试程序

```

71 public static void main(String[] args) {
72     // 测试用例
73     String[] testExpressions = {
74         "3 + 5",
75         "10 + 2 * 6",
76         "100 * 2 + 12",
77         "100 * (2 + 12)",
78         "100 * (2 + 12) / 14",
79         "2 + 5 * 6 - (6 / 3) ^ 2",
80         "(3+2) * (5-4)"
81     };
82
83     for (String expr : testExpressions) {
84         try {
85             System.out.println("The result of " + expr + " = " + calculator(expr));
86         } catch (Exception e) {
87             System.out.println("Error evaluating expression: " + expr + " - " + e.getMessage());
88         }
89     }
90 }
91 }

```

四、运行结果展示

```

D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2.4\lib\idea_rt.
The result of 3 + 5 = 8.0
The result of 10 + 2 * 6 = 22.0
The result of 100 * 2 + 12 = 212.0
The result of 100 * (2 + 12) = 1400.0
The result of 100 * (2 + 12) / 14 = 100.0
The result of 2 + 5 * 6 - (6 / 3) ^ 2 = 28.0
The result of (3+2) * (5-4) = 5.0

进程已结束，退出代码为 0

```

说明混合运算表达式程序完全正确

五、总结与收获

通过对子任务二表达式计算器的学习,我更深入地理解了使用栈数据结构解决复杂计算问题的技巧。

这个例子展示了如何利用两个栈(一个存储操作数,一个存储操作符)来实现表达式的解析和计算。通过遍历表达式字符串,识别数字和操作符,并根据运算符优先级规则进行入栈和出栈操作,最终得到正确的计算结果。

在表达式计算器的实现过程中,我也加深了对运算符优先级规则的理解,并学会了如何处理除数为零等特殊情况,提高程序的健壮性。同时,编写全面的测试用例也帮助我验证了计算器的正确性,并发现潜在的问题。

总的来说,这个例子不仅锻炼了我的算法设计和编程能力,也让我更深入地理解了如何利用栈数据结构来解决复杂的计算问题。这些经验对于我今后的问题解决和代码实现都会有很大帮助。

子任务三：

一、题目：股票最大走势区间

③寻找连续的股票最大走势区间。在股票市场中，需要识别股价走势中的连续时间段，这些时间段内股价表现出特定的行为模式，如连续上涨、连续下跌或是横盘整理等。这种分析对于技术分析尤为重要，它帮助交易者和投资者理解市场的动向，判断趋势的持续性或反转的可能性。给定一个记录股票价格的数组 `prices`，需要生成对应的 `s` 数组，`s[i]` 中记录的值为 `i-j+1`，其中 `i` 和 `j` 必须要满足 `prices[j..i-1]` 中的每一个值都符合 `prices[j] ≤ prices[i]`。`s` 数组中的最大值即为连续上涨的最大区间，如图 1 所示。

设计并实现一个算法完成对给定的 `prices` 数组上求最大连续上涨时间区间值。如果仅使用朴实的设计思想，该算法的时间复杂度和空间复杂度是多少？如果使用栈设计的算法，其时间复杂度和空间复杂度是多少？

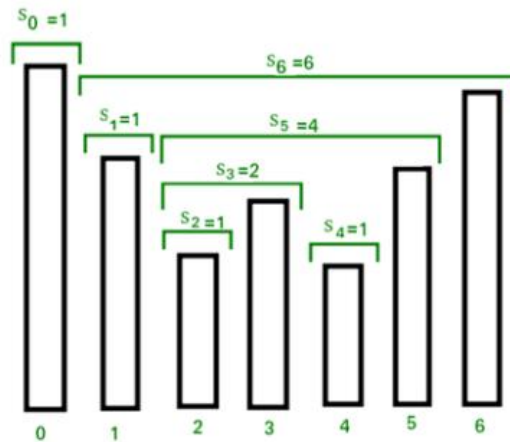


图 1：以 `prices=100,80,60,70,60,75,85` 为例的 `s` 的值

二、算法设计

栈方法：

步骤 1：初始化

创建一个栈用于存储价格的索引，创建一个数组 `s` 来存储每个位置的连续上涨区间长度。

步骤 2：遍历价格数组

对于每个价格 `prices[i]`：

栈操作：弹出栈中所有小于等于 `prices[i]` 的索引。因为这些索引对应的价格不会影响当前价格的连续上涨区间。

计算跨度：如果栈为空，说明当前价格是到目前为止的最高价，连续上涨的跨度为 `i+1`；如果栈不空，说明栈顶索引对应的价格是当前价格之前的第一个大于它的价格，跨度为 `i - stack.peek()`。

更新栈：将当前价格的索引 `i` 压入栈中。

步骤 3：返回结果

返回数组 `s`，其中每个元素 `s[i]` 表示从第 `i` 天开始的最长连续上涨区间长度。

三、复杂度分析

朴素方法：

时间复杂度： $O(n^2)$

因为对于每个元素，我们可能需要向前检查所有的元素。

空间复杂度： $O(n)$

用于存储结果数组 s 。

栈方法：

时间复杂度： $O(n)$

每个价格只会被压入栈一次并从栈中弹出一次，因此总操作数与价格的数量成线性关系。这是因为栈中存储的价格始终是严格递减的（从栈底到栈顶），每次遇到新的较大价格时，所有较小的价格都被弹出。

空间复杂度： $O(n)$

使用了一个栈来存储索引，最坏情况下所有索引都会被存入栈中。此外，数组 s 也需要 $O(n)$ 的空间来存储结果。

四、主干代码说明

（一）朴素方法

```
1 package task3of3;
2 import java.util.Stack;
3
4 public class StockPriceTrend {
5     @ public static int[] calculateSpanNaive(int[] prices) { 1个用法
6         int n = prices.length;
7         int[] s = new int[n];
8
9         for (int i = 0; i < n; i++) {
10             int span = 1;
11             for (int j = i - 1; j >= 0 && prices[j] <= prices[i]; j--) {
12                 span++;
13             }
14             s[i] = span;
15         }
16         return s;
17     }
```

（二）单调栈方法

```
19 @ public static int[] calculateSpanWithStack(int[] prices) { 1个用法
20     int n = prices.length;
21     int[] s = new int[n];
22     Stack<Integer> stack = new Stack<>();
23
24     for (int i = 0; i < n; i++) {
25         while (!stack.isEmpty() && prices[stack.peek()] <= prices[i]) {
26             stack.pop();
27         }
28         s[i] = (stack.isEmpty()) ? (i + 1) : (i - stack.peek());
29         stack.push(i);
30     }
31     return s;
32 }
```

(三) 测试类

```
34 public static void main(String[] args) {  
35     int[] prices = {100, 80, 60, 70, 60, 75, 85};  
36     int[] result2 = calculateSpanNaive(prices);  
37     int[] result1 = calculateSpanWithStack(prices);  
38     System.out.print("朴素方法: ");  
39     for (int span : result1) {  
40         System.out.print(span + " ");  
41     }  
42     System.out.println();  
43     System.out.print("栈方法: ");  
44     for (int span : result2) {  
45         System.out.print(span + " ");  
46     }  
47 }  
48 }
```

五、运行结果

```
D:\IDEA\jdk17\bin\java.exe "-j  
朴素方法: 1 1 1 2 1 4 6  
栈方法: 1 1 1 2 1 4 6  
进程已结束, 退出代码为 0
```

两种算法均实现成功

六、总结与收获

在实现寻找股票连续上涨最大区间的算法过程中,我深入理解了如何利用栈来优化时间复杂度。在最初的朴素算法中,通过双层循环遍历价格数组实现,这虽然直观但效率低下。借助栈的结构特性,我们能够有效地追踪之前的价格,并在常数时间内更新和计算当前价格的连续上涨区间。这种优化思路显著降低了算法的时间复杂度,从 $O(n^2)$ 降到了 $O(n)$ 。

通过这个任务,我不仅掌握了栈在处理序列问题中的应用,也进一步体会到合理的数据结构选择对于提升算法效率的重要性。这种栈优化方法在其他类似的区间问题中也具有广泛的应用潜力,提高了我的编程能力和算法设计思维。

任务 4：基数排序

一、题目

使用自定义的队列数据结构实现对某一个数据序列的排序(采用基数排序)，其中对待排序数据有如下的要求：

①当数据序列是整数类型的数据的时候，数据序列中每个数据的位数不要求等宽，比如：1、21、12、322、44、123、2312、765、56

②当数据序列是字符串类型的数据的时候，数据序列中每个字符串都是等宽的，比如："abc", "bde", "fad", "abd", "bef", "fdd", "abe"

注：radixsort1.txt 和 radixsort2.txt 是为上面两个数据序列提供的测试数据。

二、数据结构设计

主要数据结构：

自定义队列（Queue）：使用链表实现的队列，支持基本操作如入队（enqueue）和出队（dequeue）。

基数排序数组：整数数组和字符串数组，用于应用基数排序。

辅助数据结构：

桶（buckets）：在基数排序中，使用一个队列数组作为桶来存放特定位的数字或字符。

三、算法设计

整数排序：

根据每个数字的位数（从个位到最大位数），将数字放入相应的 0-9 桶中，然后按顺序收集这些桶中的数字。

字符串排序：

类似地，根据字符串的每个字符（从最后一个字符到第一个字符），将字符串放入相应的字母桶中，然后按顺序收集。

文件读取和写入：

读取：从文本文件读取整数或字符串序列。

写入：将排序后的数据写入指定文件。

四、主干代码说明

（一）链表节点类 Node

```
1 package task4;
2
3 public class Node<T> { 4个用法
4     public T value;
5     public Node next;
6
7     public Node(T value) { 1个用法
8         this.value = value;
9         this.next = null;
10    }
11 }
```

(二) 用链表实现队列操作

```

1  package task4;
2
3  public class Queue <T>{ 8个用法
4      Node head,tail; 7个用法
5  >   public Queue() { this.head = this.tail = null; }
6
7
8
9  >   public boolean isEmpty() { return head == null; }
10
11
12
13   public void enqueue(T value){ 2个用法
14       Node temp = new Node<>(value);
15       if(this.tail == null){
16           this.head = this.tail = temp;
17           return;
18       }
19       this.tail.next = temp;
20       this.tail = temp;
21   }
22
23
24   public T dequeue(){ 2个用法
25       if(isEmpty()){
26           System.out.println("This Queue is Empty");
27           return null;
28       }
29       T temp= (T) head.value;
30       this.head = this.head.next;
31       if(head == null){
32           this.tail = null;
33       }
34       return temp;
35   }
36 }

```

(三) RadixSort

```

1  package task4;
2
3  import java.io.*;
4  import java.util.Scanner;
5
6  public class RadixSort { 1个用法
7
8      //数字数据基数排序
9  @   public static void radixSortForInteger(int[] arr) { 1个用法
10
11       //判断最大数字是几位数字
12       int max = Integer.MIN_VALUE;
13       for (int value : arr) {
14           if (value >= max) {
15               max = value;
16           }
17       }
18       int maxLength = Integer.toString(max).length();
19
20
21       //0~9 一个数字一个桶
22       Queue<Integer>[] buckets = new Queue[10];
23       for(int i = 0; i < buckets.length; i++)
24           buckets[i] = new Queue<Integer>();

```

```

27 //根据最大位数决定比较次数, 从个位数开始比较, 依次将其放入对应桶中
28 for (int i = 0, n = 1; i < maxLength; i++, n = n * 10) {
29     for (int value : arr) {
30         //求对应位数应放入的桶
31         int num = (value / n) % 10;
32         buckets[num].enqueue(value);
33     }
34     int index = 0;
35     for (Queue queue : buckets) {
36         while (!queue.isEmpty()) {
37             arr[index] = (int) queue.dequeue();
38             index++;
39         }
40     }
41 }
42
43 }
44
45 //将结果输出到指定文档中
46 public static void writeToFile1(int[] array, String filePath) throws IOException { 1个用法
47     try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
48         int index=0;
49         for (int value : array) {
50             writer.write( str: value + " ");
51             index++;
52             if(index==10){
53                 writer.write( str: "\n");
54                 index=0;
55             }
56         }
57     }
58 }

```

```

60 public static void writeToFile2(String[] array, String filePath) throws IOException { 1个用法
61     try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
62         int index=0;
63         for (String value : array) {
64             writer.write( str: value + " ");
65             index++;
66             if(index==10){
67                 writer.write( str: "\n");
68                 index=0;
69             }
70         }
71     }
72 }

```

```

74 //字符串数据基数排序
75 public static void radixSortForString(String[] strings) { 1个用法
76
77     //求取最大长度
78     int maxLength=strings[0].length();
79
80     //为大小写共 52 个字母创建桶
81     Queue[] temp = new Queue[52];
82     for(int i = 0; i < temp.length; i++)
83         temp[i] = new Queue<String>();
84
85     for (int i = 0; i < maxLength; i++) {
86         for (String string : strings) {
87             char c = string.charAt(maxLength - 1 - i);
88             int num = c >= 'a' ? c - 'a' + 26 : c - 'A';
89             temp[num].enqueue(string);
90         }
91         int index = 0;
92         for (Queue queue : temp) {
93             while (!queue.isEmpty()) {
94                 strings[index] = (String) queue.dequeue();
95                 index++;
96             }
97         }
98     }
99
100 }

```

```

102 //两个测试函数用来测试是否已经排序成功
103 @ public static boolean isSortedOfNums(int[] nums){ 1个用法
104     for(int i = 0; i < nums.length - 1; i++){
105         if(nums[i] > nums[i+1]){
106             return false;
107         }
108     }
109     return true;
110 }
111 @ public static boolean isSortedOfStrings(String[] strings){ 1个用法
112     for(int i = 0; i < strings.length - 1; i++){
113         if(strings[i].compareTo(strings[i+1]) > 0){
114             return false;
115         }
116     }
117     return true;
118 }
119
120 //补上两个读取数据的函数
121 @ public static String[] getStrings(String filename) throws FileNotFoundException { 1个用法
122     Scanner in= new Scanner(new File(filename));
123     StringBuffer sb=new StringBuffer();
124     String temp=in.next();
125     sb.append(temp);
126     temp=in.next();
127     while(!temp.equals("stop")){
128         sb.append('\n');
129         sb.append(temp);
130         temp=in.next();
131     }
132     return sb.toString().split( regex: "\n");
133 }

```

```

136 @ public static int[] getNums(String filename) throws FileNotFoundException { 1个用法
137     Scanner in=new Scanner(new File(filename));
138     StringBuffer sb=new StringBuffer();
139     String temp=in.next();
140     sb.append(temp);
141     temp=in.next();
142     while(!temp.equals("stop")){
143         sb.append('\n');
144         sb.append(temp);
145         temp=in.next();
146     }
147     int[] nums= new int[sb.toString().split( regex: "\n").length];
148     for(int i = 0; i < nums.length; i++){
149         nums[i]=Integer.parseInt(sb.toString().split( regex: "\n")[i]);
150     }
151     return nums;
152 }
153
154 }

```

(四) 读取文件

```

1 package task4;
2
3 import java.io.IOException;
4
5 import static task4.RadixSort.*;
6
7 public class Test {
8
9     //文件地址
10    public static void main(String[] args) throws IOException {
11        int[] nums = getNums( filename: "radixSort1.txt");
12        String[] strings = getStrings( filename: "radixSort2.txt");
13        radixSortForInteger(nums);
14        writeFile1(nums, filePath: "outputIntegers.txt");
15        System.out.println(isSortedOfNums(nums) ?
16            "The numbers have been sorted!" : "The numbers are not sorted!");
17        radixSortForString(strings);
18        writeFile2(strings, filePath: "outputStrings.txt");
19        System.out.println(isSortedOfStrings(strings) ?
20            "The strings have been sorted!" : "The strings are not sorted!");
21    }
22 }

```

五、运行结果展示

```
The numbers have been sorted!
The strings have been sorted!
```

整数排序后：

```
0 6 10 16 18 18 21 23 27 28
33 37 37 37 41 42 43 47 48 51
56 68 69 70 85 87 91 96 97 102
105 111 112 124 126 128 133 140 142 142
142 146 147 148 149 154 165 166 169 175
175 178 188 200 200 202 207 215 217 221
229 236 236 237 255 263 264 264 265 270
271 277 279 280 282 286 289 296 298 299
308 313 314 317 319 319 322 324 326 335
339 343 343 345 354 359 360 361 362 363
363 363 368 368 377 379 384 384 386 392
395 404 413 414 414 418 419 435 436 438
439 442 444 445 447 449 457 457 463 466
468 473 476 478 483 492 497 500 514 520
528 533 535 550 552 553 554 560 563 567
574 575 575 576 577 588 590 600 603 614
615 617 618 637 638 643 643 653 654 657
657 660 664 668 671 692 695 714 720 724
731 735 736 737 743 744 746 753 753 759
764 771 783 786 792 798 798 800 802 803
805 813 814 824 829 838 843 862 865 869
874 875 877 881 882 882 887 891 891 895
900 902 908 909 909 912 913 916 936 936
937 939 944 946 947 951 953 961 964 967
967 968 972 974 975 975 977 985 985 985
990 997 998 1000 1002 1003 1017 1018 1019 1020
1031 1031 1033 1037 1041 1050 1054 1058 1063 1070
1077 1079 1082 1084 1084 1086 1087 1087 1089 1096
1097 1098 1100 1103 1103 1109 1114 1127 1130 1132
1138 1148 1155 1159 1161 1162 1164 1164 1174 1181
1185 1186 1186 1187 1189 1212 1213 1216 1219 1222
1224 1227 1228 1230 1231 1236 1239 1240 1241 1244
1274 1282 1284 1293 1295 1296 1297 1307 1309 1313
1321 1330 1332 1332 1339 1339 1342 1343 1345 1347
1349 1350 1359 1361 1363 1367 1380 1381 1381 1385
1395 1398 1404 1406 1407 1409 1412 1424 1426 1427
```

.....

```
1084509 1084546 1086065 1086809 1087383 1087683 1088488 1088631 1089003 1089956
1090201 1090972 1092024 1092189 1092692 1092927 1093078 1094687 1095673 1097685
1100058 1100687 1101311 1102232 1103190 1103453 1103650 1104039 1105720 1108402
1110509 1111005 1111639 1113346 1113425 1114002 1114106 1115015 1118243 1118535
1118981 1122375 1122440 1124109 1124240 1124401 1124560 1124960 1125876 1128908
1128976 1129114 1129244 1129577 1129869 1130717 1132250 1133015 1134215 1134542
1136467 1137253 1137502 1138252 1138498 1138808 1139237 1140000 1140217 1141194
1142686 1145847 1146680 1147051 1147200 1147608 1148092 1148364 1148910 1149127
1149481 1149689 1150093 1150142 1150798 1152640 1152970 1153389 1153483 1153878
1153943 1154110 1156861 1161139 1161462 1162076 1162342 1163524 1164479 1164499
1164561 1164718 1165166 1165534 1167366 1168040 1169896 1170951 1171260 1173378
1173408 1174101 1176164 1177414 1178092 1178337 1179383 1179499 1180574 1181343
1181623 1183479 1183684 1184024 1184391 1184917 1188641 1188971 1189185 1189582
1189753 1190630 1191226 1191454 1191564 1192946 1194653 1195465 1195705 1196882
1198179 1199314 1199819 1200850 1201068 1201257 1202775 1203380 1203540 1203580
1203599 1204609 1207368 1207500 1207957 1210087 1210693 1211032 1213413 1214466
1214714 1214914 1216408 1217347 1218801 1219176 1219773 1220457 1221077 1221546
1221558 1221831 1222154 1222410 1222540 1222578 1223702 1224240 1224891 1225525
1225955 1226036 1227416 1228735 1228800 1230213 1230231 1231992 1233870 1234264
```

字符串排序后:

```
ACgnsNit ADBJMSzB ADRKabPw ADRKsb1B ADjSFRsM AEppMxnC AGIWmdlX AKpTKsKE ALPQRdWg AP0eBcfB
AQdwNrjL ARzprbYP AUNkreFd AwtdOqOs AXQvcSbs AVOGHDOX AYbuiMYA AYvhbcNk AanVzwtS AgYNUvcd
AlRpOQFn AlOcZkRl AmpCpEzH ApyjWZtA AqcsZvdQ AsasKDXt AuAkCOZp AyeMQadX AzbxOQJz AztMmxSg
BB0JEHQ0 BCKKRHgK BCVEDsin BEeMSeCF BFPClXne BFsZzJdJ BGBPBCFy BHXvhnwU BIaGgssZ BIpcyQtC
BKthFwAh BLSapdFF BLpdDeyE BLrxGkDU BMSgeojg BNDToivF BNoQwkio BPEewCuy BQLpANqw BRdegocn
BRvuBbJk BSvePbmL BVowzoaV BXsNOPsc BYExhMvV BZROZEhw BZWGzKSb BZatoZIY BcfSsGII BdldiaVx
BeESGied BeNiNNeP BeVlGzDR BeBCMCFG BFQXQRsQ BhdMwoqp BjIpWzKO BlMgPzth BldNMuoT BluZbTNF
BmA0dcVg BmwJuUBP BnJDKKnP BowLatbX BoqxerFF BpLkTynI BqUXvyKK BrOWZIQJ Br1jOWPU BrwMILAt
BsMzLVwB BsOLVcXt BTIZCAIO BtPvaogP BwKtXaWi BzwqTzbP CBRRnnMj CHYPClDu CKFFxely CKzDQoKU
CNTVdFNQ COFaEqsA CQXRSMYS CSNLVWRY CwjPhsZI CZGygyfY CbjEjRiS CbkthTGB ChVrQQNP ChYqdePX
CpUMDokm CsOLlPXE CsReToVT CtyoeBif CuYqXlxc CvVFDJGp CxwtNrcj CzUcMLbQ CzYvaaoji DAieCqpw
DAwHxUII DECHGgPr DERGcsXW DGEJHqLJ DJAlTpBp DJVEqGKh DNEHJNNH DOIzzAIY DQpJSjFZ DSDJEFTl
DTGETGoI DUmmGmkV DVIUADcP DVrdTWAG DYYJZWot DZpvQiND DcuwFuDM DdXkQqWb DdbKZMDY DeQbsPHB
DfiAuKcs DhBCTDdg DhJkthfw DhXNmMR DlbXlSnC DlnBLKaO DmRlOlwm DnQENUKB DoMuQsvu DoPlQappt
DQojhoSA DotyEsrg DqMuGCEr DqfgvmYH DrOcfQgB DSAuykiF DvTjTKqs DxoBFntt DyFEruoL EbfyRheE
EBUAWzJK EBzwtuMw EDbqMzSi EDdYgnPy EdqtfbOB EFmHqVlW EHPZSHQd EIfvFvFB EnIZuTza EpJzRNKv
EKcuHzqc EOJntupu EPVRgPsZ EQLOjPpF ERPmqnsu ETLLeOJFX ETLkqTdN EUwmjXbc EVJfyFIL EVsWYzAH
EVzZBZzk EYyJGsyL EYgwhHAW EYZBBKvY EaQGvbGp EavcnuJP EbfGvKAK EbhwbXKm EdMuaVYD EeVKdyDj
EgJSaAA EhwUEJTY EilwMEPB ElqPuFvK EmNnwXNS EmwIwMDL EmfZTCBY EnIZuTza EpJzRNKv FCcrHYuC
ErMskxET EsCgaPrL EuiVkpSF EvUmGRDK ExQSCffk ExTgEnDT EyVBnyXu EyXXtZNf FCCrHYuC FCwTVUAF
FDgoZVjN FEXMDGUb FFDsrSzo FFnKjQdk FGIOvVOW FHTidlBg FHTlpsYc FHWtFr1Z FIsIKULW FK0gQcPo
FKyEBPKm FMGZAay FMzPHWAd FOlvRYDO FOvLcbrt FPahkMue FQXBjKpQ FXGZDnhz FYbtPatY FZWZHTZD
FZYPLNzB FZeedXhF FZjCpirc FarfTEoH FcBidqcy FyOAIWu FkQlOfbd FliabauB Flzbauxy Flzbauxy
FpKtIymA FqRPXuNQ FuiyaYQf FvLybtNi FvUMRDJb FvzQlMfY FxSEupcd FzPLBzSG FzYBLtbd GCAEFLLk
GCDKlEdF GCSBZHTY GCaZWpgj GFBsWBPD GIQTmyzf GJDFwmVn GJJnbHEN GKCrVZLk GLJmJhMi GMfkNdth
GODXKHZs GOTSkgMU GRJDeiZB GRhiGRFe GSFReRbe GUUyyJLR GvCEcmFf GYskJRfk GzoUZcdp GbDmlrQS
GbTatKSK GcGcwuhr GdOdMtZE GeQlStiv GjLPfZHC GkBrkYom GkTanKjv Gknhihhs GmYBDNc GmioEHXJ
GoHjvKpW GocDeEwt GqQzoPeT GsLZqbvw GsQvOpmv GSxYZng GsqYNBis GvCchkeU GvDeIXPA GveBXjaT
GwGMPTAV GwOvXkwx GxAeJGwe GxDrHrrw Gx1Kpiu GxtFTxoz GyjtdFzN HArnkdeU HDLDYAFB HDLDFYAFB
HITKjVWQ HIsnaERr HLSojusj HMGRFufh HPBUhuxS HPMhBTme HPGGuqCop HrdwZnvS HSuLgxxR HTTfskel
HVRgSQaT HYCxLykW HYGCVEPI HYarFmDN HZKTnVPH HZUGIHGR HaAHNCEq HaxQWgji HbHesScN HbWKhIqv
HdSremou HdTbOYhD HfCZgMae HflvPYeo HgJnRIux HHuHmOmy HhkxyXgc HijKukta HioUtMXo HlaeevsN
HqOFVOuy HraqLdVW HsBcxRwD HwCvDotb HwSJcmr1 HyRiZEIz IBwtUPPi ICdJqTLW IDOOvsnx IJDRaglH
IJWvKNTy ILSQxwMW ILpRfpkF INosAxVq IPeWnUrX IPhoYJHQ IQhAvhRZ ISfNgsgo ITDwMXng IUWzXXFI
IUsmJNmV IWYDockX IYDuHJTV IYDyQiiu IYsojrWE IZDpTHfE IbfAmXhx IcgDWCoP IeIJGRsv IeQYwSxn
IfVwYlEG IibBuPXg IkFtfrdJ ImOieFnJ IoaXfJBQ IoZlztvx IrTwtgYw Itl1JfXQ IwQymbJt IdXHJHJm
IxSCgfaF IxmGKRHW IyCJOHsn IyJQucUq Iygmduxj JAhoYuro JFXmNqxa JGERhkkB JGXBKRbf JImqPIJC
JInBshBN JJYOpAcz JKEyjkNm JKsxhcDj JNWLldtF JNoiUpib JQiqnxAm JQmrgLxR JgttvGgj JjIgbYQV
JJCvZbYy JJXQSWHqT JYKibvSO JYRCrtkt JdugyOeC JeDuPySx JePJJcsB JgttvGgj JjIgbYQV JjKasBzoQ
JnsnmYyu JqIYooUJ JqchfsDZ JrZmDQlI JrpbqGRq JuQJPSCb KABdlLVb KBTZudej KBfionGw KCYVMLWH
KCiAPHib KCrCRFjv KCuveSUF KCwIQzZn KDCLMZVm KDIftNu1 KDfdtRbK KEgiBXDm KFRJzBDe KIqHgtMh
KJTeefJQ KKCEzlpg KKAxCTAX KLFznHfw KNAUOpnA KRUEapns KSCyQJnd KTtGDeEK KUQQZpDs KVsUWRKL
KZBBbVUP KZKqdilZ KZgZrDMt KaekRHwM KbwIWB1I KcLSAowQ KdIYgrNk KgaBokeB KhiLiKlk KiGrCjon
Kiwnxqgh KkdztITW KlGrckDy KoYzMNCH KptuyFbk KsIDaqCT KuvxNKdC KvZmrWEO KxwWujTk KyNcTDbb
KyPuyQGW KzJQihZu LAaCKJA LCxFONXR LELMxJeP LERApwIj LFeZaQCa LHdJbXKG LIfvKBcc LIgKsqzC
LLAbZdJD LLPhVqi LOQWguBi LQtMFMf LRFNFSsp LsUzTcd LvvZaQtE LwbDuzi LwPNYDmQ LzREDklK
LcnvjAaN LdSLGvY LeGQlVsY LgfMnLwx LgssCyEj LiZKtGwt LjzJCWCN LmZJZmRO LmlpqFbd LqJPSzWA
LrPhCkqt LriOKwMM LtLAVSTu LvouSyGU LvttCiCL LyQHCJv1 LzjJfYmk MAMORPiQ MEwo1reb MFGSUTKr
MFOersyW MHcWvKCu MHVjiFsL MLChYbry MLRErnmE MLaOubIB MMISFRwv MmicBQtn MoyTcIQJ MPgAQRLS
MTZePcsc MZgIleU MUgoEwDz MUMCIUTf MUMonznM MXmqANRY MahpYhZQ MdTdmJEh MeTltd1T Mewlynpc
MfDcfEue MgnSCSia MhZYVDaE Mh1OPeyo MIAFYFyt M1UrcRwL MkhnRFLl MkndsLPC MpIusXUP MphjXhFy
MqkZYvWV MsmobpTm MtXKibOE MtjQXZUu MuqISGbj MvbiGdaQ MvgYAHVY MxHMBtsQ MyCFEGyX MygtEtot
NCEycQhm NDFbvreI NDceLv1z NFzQzzvw NGaYRtGc NHIAiYUA NHsonMMf NIHXHivM NJzYSMeL NKUqhWw
```

.....

```
ybzBiXoT yedDKlXK yhhhUmFZ yizYwiMC ykEgUsRb ympNuhHq ynumxUjF yqBvFgfo yqNbJJvp yqRzxmzw
ysbTEeic ysualdEX ysyceUPZ ytVxNvTu yvAoheKA yvFACzzZ ywubSoqX yx1k1lHT yyQtFXvm yzthZHxm
yzuRrjUp zAP1ZGHC zAhjQveW zAlDCdbN zBXpCARs zDvJ0oRI zE1zxSXB zFKxUWQM zFzCSckv zIxEOCBQ
zJGuBHZt zKgtGunP zKjoVYwz zKohVqQR zLWHYmeQ zLXFseML zLcAlmIx zMMTWdhx zOWHMzJD zOnAJYqj
zQCRFyWz zQPClTub zRLiouoj zSNcqCDk zTDZiQuz zUwIWSML zVFWvili zWsvWMTz zXBRmOfJ zXWmZNRr
zYNPcfoS zZfrYBW zZgkXqgt zbJNVRKU zbVBOAXn zcjHhskB zdROHPxm zdzRKwXO zeEazNEa zelaQmtc
zfrVOagj zIERTCDH zJdxuOGn zjhZTuac zjwiYEqc zthWBwXw zvrAVQls zwLTmXHN zyCbelhX zyknRWGz
```

六、总结与收获

这个实验提供了实践自定义数据结构（如队列）和复杂排序算法（如基数排序）的机会。通过这个题目，我们不仅学习了基数排序算法的原理和实现，还加深了对文件操作、数据结构和算法的综合应用的理解。此外，实验还提高了对不同数据类型处理的灵活性，以及如何根据数据特性选择合适的排序算法。通过这个实验，我们提升了编程技巧和解决实际问题的能力。